

Universidad Politécnica de San Luis Potosí

Programación Web II

Lenguaje PHP, HTML, CSS
Programación Web

Consulta de kardex Manual del técnico

Asesor del proyecto Liliana
Gámez Zavala

Contenido

1. Introducción	3
2. Inicio del programa	3
2.1 Inicio de sesión	3
2.2 Verificación de sesión activa	3
2.3 Asignación del rol del usuario	4
2.4 Redirección según el rol del usuario	4
2.5 Aplicación en la barra de navegación	4
2.6 Login.php	4
2.7 Login_control.php	5
3. Pagina A.php	7
3.1 Registrar.php	8
3.2 Consulta.php	12
3.3 Modificar.php	14
3.4 modificarUsuario.php	15
3.5 ConsultaPlan.php	15
3.6 Importación de Kardex	18
3.7 Importación de plan de estudios	20
4. Ayuda.....	23
5. Estadistica control.php	25
6. PDF Estadísticas estudiantes	26
6.1 PHP.....	26
7. Reporte.php	36
8. Generación de Kardex en PDF	38
8.1 PHP.....	38
9. Verkardex.php	46
10. Kardex del alumno	48
10.1 PHP	48
10.2 CSS	55
11. Pagina alumno.php	58
12. Página de maestros	61
12.1 HTML.....	61
12.2 CSS	63
13. Styles	66
1. Admin.css	66
14. Conclusiones	68

1. INTRODUCCION

Este manual técnico documenta el desarrollo de un sistema web dinámico enfocado en la gestión de usuarios. El sistema incluye funcionalidades clave como la autenticación de usuarios, modificación de datos en tiempo real y una interfaz adaptable para garantizar una experiencia de usuario fluida en diferentes dispositivos.

El proyecto combina PHP para la lógica del servidor, MySQL para la gestión de datos y CSS para el diseño visual. Adicionalmente, se implementan principios de seguridad como validación de roles de usuario y control de acceso, así como prácticas de diseño moderno que mejoran la interactividad y la estética.

Este manual se organiza en secciones que explican cada fragmento de código en detalle, describiendo su propósito, lógica de implementación y cómo interactúa con otras partes del sistema. De esta manera, cualquier programador podrá entender el sistema, personalizarlo y adaptarlo a sus necesidades.

2. INICIO DEL PROGRAMA

Este sistema asegura que solo los usuarios autenticados y con el rol adecuado puedan acceder a las áreas correspondientes del sitio.

2.1 Inicio de sesión

```
1 <?php
2 session_start();
3
```

- Esta línea inicia la sesión PHP. Es necesario llamarla al comienzo de cualquier archivo que quiera acceder a variables de sesión.

2.2. Verificación de sesión activa

```
4 // Verifica si la sesión está iniciada
5 if (!isset($_SESSION['tipo'])) {
6     header('Location: https://orangered-donkey-303339.hostingersite.com/proyectoFinal/login.php');
7     exit();
8 } else {
9     $role = $_SESSION['tipo'];
10 }
11
```

- Se verifica si la variable de sesión tipo no está configurada (lo que significa que el usuario no ha iniciado sesión).
- Si la variable no existe, el usuario es redirigido a la página de inicio de sesión (login.php).

- El uso de `exit()`; asegura que el script se detenga inmediatamente después de la redirección.

2.3. Asignación del rol del usuario

`$role = $_SESSION['tipo'];`

- Si la sesión está activa, se obtiene el valor de la variable tipo almacenada en la sesión y se asigna a la variable `$role`.

2.4. Redirección según el rol del usuario

```

11
12 // Determina la URL de inicio según el rol
13 if ($role == '0') {
14     $inicioUrl = '../paginaA.php';
15 } else {
16     if ($role == '1') {
17         $inicioUrl = '../paginaM.html';
18     } else {
19         header('Location: https://orangered-donkey-303339.hostingersite.com/proyectoFinal/archivosPHP/acceso_denegado.php');
20         exit();
21     }
22 }

```

Si el usuario tiene un rol '0', se define que su página inicial será `paginaA.php`.

Si el usuario tiene un rol '1', su página inicial será `paginaM.html`.

Si el rol no corresponde a ninguno de los anteriores, se considera no autorizado, y el usuario es redirigido a una página de acceso denegado (`acceso_denegado.php`). El uso de `exit()`; asegura que el script no continúe después de esta redirección.

2.5. Aplicación en la barra de navegación

```

<!-- Barra de navegación -->
<nav class="barra">
    <ul class="topnav">
        <li><a href="<?php echo $inicioUrl; ?>">Inicio</a></li>
        <li><a href="../archivosPHP/logout.php">Logout</a></li>
        <li class="right"><a href="../archivosPHP/ayuda.php">Help</a></li>
    </ul>
</nav>

```

`<li><a href="<?php echo $inicioUrl; ?>">Inicio</a></li>`

La redirección es procesada aplicando la variable en el enlace de la barra de navegación para el apartado home,

2.6 Login.php

1. El usuario accede a la página de inicio de sesión.
2. Si hubo un error previo en el inicio de sesión, se muestra un mensaje en la parte superior del formulario.
3. El usuario ingresa sus credenciales y las envía al servidor.

4. El servidor procesa los datos en login_control.php y redirige según corresponda:

- Credenciales Correctas: Acceso al sistema.
- Credenciales Incorrectas: Redirige de vuelta al formulario con un mensaje de error.

Este flujo garantiza que solo usuarios autenticados puedan ingresar al sistema.

- Manejo de errores con php

Funcionalidad:

- Inicio de Sesión: Se inicia la sesión para acceder a las variables de sesión disponibles.
 - Obtener el Mensaje de Error: Se verifica si existe una bandera de error activa (error_flag). Si está activa, se recupera el mensaje de error (error_message) para mostrarlo al usuario.
 - Limpiar el Error: Una vez recuperado, tanto el mensaje como la bandera se eliminan usando unset() para que no persistan en sesiones futuras.
- Estructura html
 - Formulario de Inicio de Sesión: El formulario utiliza el método POST para enviar los datos de usuario y contraseña a login_control.php.
 - Campos requeridos para el usuario y la contraseña.
 - Manejo de Errores: Si existe un mensaje de error (\$error_message), este se muestra en un cuadro de alerta (alert-danger) de Bootstrap.
 - Diseño de la Página: Se divide en dos secciones principales:
 - Formulario de Inicio de Sesión.
 - Sección de Bienvenida: Un mensaje introductorio y el logotipo de la institución.

2.7 Login_control.php

Este script en PHP maneja el proceso de autenticación de usuarios mediante una combinación de sesiones, validación de credenciales y redirección según el tipo de usuario. El sistema utiliza consultas preparadas para proteger contra inyecciones SQL y emplea password_verify para comparar contraseñas de manera segura.

```

1  <?php
2  session_start();
3  $bandera = FALSE;
4
5  // Verificar si se realizó la solicitud POST
6  if ($_SERVER["REQUEST_METHOD"] == "POST") {
7      $usuario = trim($_POST['usuario']);
8      $password_user = trim($_POST['password']);
9
10     if ($usuario != "" && $password_user != "") {
11         // Conexión a la base de datos
12         $conn = require("conexion.php");
13
14         if ($conn->connect_error) {
15             $_SESSION['error_message'] = "La conexión a la base de datos no se pudo realizar";
16             $_SESSION['error_flag'] = true;
17         } else {
18             // Usar consultas preparadas para evitar inyecciones SQL
19             $stmt = $conn->prepare("SELECT usuario, password, id, tipo FROM usuarios WHERE usuario = ?");
20             $stmt->bind_param("s", $usuario);
21             $stmt->execute();
22             $result = $stmt->get_result();
23
24             if ($result->num_rows > 0) {
25                 $row = $result->fetch_assoc();
26
27                 // Verificar la contraseña con password_verify
28                 if (password_verify($password_user, $row['password'])) {
29                     $_SESSION['id'] = $row['id'];
30                     $_SESSION['usuario'] = $row['usuario'];
31                     $_SESSION['tipo'] = $row['tipo'];
32
33                     exit();
34
35                     case 1:
36                         header("Location: https://orangered-donkey-303339.hostingersite.com/proyectoFinal/paginaM.html");
37                         exit();
38                     case 2:
39                         header("Location: https://orangered-donkey-303339.hostingersite.com/proyectoFinal/paginaAl.php");
40                         exit();
41
42                 }
43             } else {
44                 $_SESSION['error_message'] = "Contraseña incorrecta.";
45                 $_SESSION['error_flag'] = true;
46                 header("Location: https://orangered-donkey-303339.hostingersite.com/proyectoFinal/login.php");
47                 exit();
48             }
49         } else {
50             $_SESSION['error_message'] = "Usuario no encontrado.";
51             $_SESSION['error_flag'] = true;
52             header("Location: https://orangered-donkey-303339.hostingersite.com/proyectoFinal/login.php");
53             exit();
54         }
55     }
56
57     $stmt->close();
58     $conn->close();
59
60 } else {
61     $_SESSION['error_message'] = "Usuario o contraseña incorrectos.";
62     $_SESSION['error_flag'] = true;
63     header("Location: https://orangered-donkey-303339.hostingersite.com/proyectoFinal/login.php");
64     exit();
65 }
66 }
67 }
68 ?>

```

Funcionamiento:

- Validación de Método POST: El script verifica si la solicitud fue enviada mediante POST, asegurando que los datos no sean accesibles a través de otros métodos.
- Consulta y Verificación de Credenciales
- Gestión de Sesiones y Redirección: Según el tipo de usuario, se establecen variables de sesión y se redirige al usuario a una página específica.

- Manejo de Errores: Los errores de conexión, usuario no encontrado o contraseña incorrecta se gestionan utilizando mensajes almacenados en la sesión y redirigiendo de vuelta al formulario de inicio de sesión

3 PAGINA A.PHP

La página de administrador está protegida mediante validación de sesión y roles. Solo los usuarios con rol de administrador (tipo 0) tienen acceso.

El código está estructurado para garantizar seguridad, modularidad y claridad en su funcionamiento.

La interfaz está dividida en tres áreas principales: Administración de Usuarios, Plan de Estudios, y Kardex. Cada una contiene enlaces a acciones específicas.

Directorios:

- Directorio archivosPHP/: Contiene scripts para manejo de datos.
- Directorio importar/: Formularios para subir archivos relacionados con el kardex y planes de estudio.
- Directorio styles/: Archivos CSS para personalización visual.

```
1 <?php
2 session_start();
3
4 // Verifica si la sesión está iniciada
5 if (!isset($_SESSION['tipo'])) {
6     // Si no está iniciada la sesión, redirige al login
7     header('Location: https://orangered-donkey-303339.hostingersite.com/proyectoFinal/login.php');
8     exit(); // Detiene la ejecución del script para evitar que se siga ejecutando el resto del código
9 } else {
10     $role = $_SESSION['tipo']; // Obtén el rol del usuario desde la sesión
11 }
12
13 // Verifica si el rol no es 0 (no tiene permiso para acceder)
14 if ($role != '0') {
15     header('Location: https://orangered-donkey-303339.hostingersite.com/proyectoFinal/archivosPHP/acceso_denegado.php');
16     exit();
17 }
18 ?>
19
20
21 <!DOCTYPE html>
22 <html lang="es">
23 <head>
24     <meta charset="UTF-8">
25     <meta name="viewport" content="width=device-width, initial-scale=1.0">
26     <title>Administrador</title>
27     <link rel="stylesheet" href="./styles/navM.css"> <link rel="stylesheet" href="./styles/paginaMaestros.css">
28 </head>
29 <body>
30     <!-- Barra de navegación -->
31     <nav class="barra">
32         <ul class="topnav">
```



```

35     <li class="right"><a href="./archivosPHP/ayuda.php">Help</a></li>
36 </ul>
37 </nav>
38
39 <header>¡Bienvenido a la pagina de administrador!</header>
40
41 <!-- Contenido principal -->
42 <div class="card-container">
43     <!-- Sección 1 -->
44     <div class="card">
45         <h5>ADMINISTRACIÓN DE USUARIOS</h5><br>
46         <a href="registro.html">Alta de usuarios</a><br><br>
47         <a href="./archivosPHP/bajaUsuario.php">Baja de usuarios</a><br><br>
48         <a href="./archivosPHP/consulta.php">Consulta de usuarios</a><br><br>
49         <a href="./archivosPHP/modificar.php">Modificar usuario</a><br><br>
50     </div>
51     <!-- Sección 2 -->
52     <div class="card">
53         <h5>PLAN DE ESTUDIOS</h5><br>
54         <a href="./importar/planEstudios.html" class="accion">Alta de plan de estudios</a><br><br>
55         <a href="./archivosPHP/consultaPlan.php" class="accion">Consulta de plan de estudios</a><br><br>
56     </div>
57     <!-- Sección 3 -->
58     <div class="card">
59         <h5>KARDEX</h5><br>
60         <a href="./importar/kardex.html" class="accion">Importar KARDEX</a><br><br>
61         <a href="./archivosPHP/verKardex.php" class="accion">Ir a Kardex</a><br><br>
62         <a href="./archivosPHP/reporte.php" class="accion">Consulta Reporte de alumno</a><br><br>
63         <a href="./archivosPHP/estadistica_control.php" class="accion">Estadísticas de estudiante</a><br><br>
64     </div>
65 </div>
66 </body>
67 </html>

```

3.1 Registrar.php

Este módulo de registro asegura que solo los administradores (rol 0) puedan acceder al formulario, validando la sesión y el rol del usuario. El formulario es dinámico, adaptándose según el tipo de usuario; por ejemplo, los alumnos deben ingresar una matrícula de 6 dígitos. Las contraseñas se encriptan usando password_hash antes de ser almacenadas en la base de datos, garantizando seguridad. L..l{p0

```

1  <?php
2  session_start();
3
4  // Verifica si la sesión está iniciada
5  if (!isset($_SESSION['tipo'])) {
6      // Si no está iniciada la sesión, redirige al login
7      header('Location: https://orangered-donkey-303339.hostingersite.com/proyectoFinal/login.php'); // Cambia 'login.php' por el nombre de tu página de login
8      exit(); // Detiene la ejecución del script para evitar que se siga ejecutando el resto del código
9  } else {
10     $role = $_SESSION['tipo']; // Obtén el rol del usuario desde la sesión
11 }
12
13 // Verifica si el rol no es 0 (no tiene permiso para acceder)
14 if ($role != '0') {
15     header('Location: acceso_denegado.php'); // Redirige a una página de acceso denegado
16     exit();
17 }
18
19 if (isset($_GET['role'])) {
20     $role = $_GET['role'];
21 }
22
23 <!DOCTYPE html>
24 <html lang="en">
25 <head>
26     <meta charset="UTF-8">
27     <meta name="viewport" content="width=device-width, initial-scale=1.0">
28     <link rel="stylesheet" href="../styles/admin.css">
29     <link rel="stylesheet" href="../styles/navM.css">
30     <title>Registros</title>
31 </head>
32 <body>
33     <!-- Barra de navegación -->

```



```

34 <!-- barra de navegación -->
35 <nav class="barra">
36   <ul class="topnav">
37     <li><a class="active" href="../paginaA.php">Inicio</a></li>
38     <li><a href="logout.php">Logout</a></li>
39     <li class="right"><a href="ayuda.php">Help</a></li>
40   </ul>
41 </nav>
42 <div class="contenido">
43   <div class="caja">
44     <div class="tipo">
45       <center>
46         <h1>Formulario de Registro</h1>
47         <form action="registrar.php" method="post">
48           <input type="hidden" name="tipo" value="<?php echo $role; ?>">
49           <?php if ($role == "2") { ?>
50             <label for="usuario">Usuario (Matrícula):</label><br>
51             <input type="text" name="usuario" id="usuario" pattern="\d{6}" title="Debe ser una clave de 6 dígitos" required><br>
52           <?php } else { ?>
53             <label for="usuario">Nombre de usuario:</label><br>
54             <input type="text" name="usuario" id="usuario" required><br>
55           <?php } ?>
56
57           <label for="name">Nombre completo:</label><br>
58           <input type="text" name="name" id="name" required><br>
59
60           <label for="password">Contraseña:</label><br>
61           <input type="password" name="password" id="password" required><br>
62
63           <label for="email">Correo electrónico:</label><br>
64           <input type="email" name="email" id="email" required><br>
65
66           <button type="submit" class="accion">Registrar</button><br>
67
68           <input type="password" name="password" id="password" required><br>
69
70           <label for="email">Correo electrónico:</label><br>
71           <input type="email" name="email" id="email" required><br>
72
73           <button type="submit" class="accion">Registrar</button><br>
74         </form>
75
76         <br><button onclick="window.location.href='https://orangered-donkey-303339.hostingersite.com/proyectoFinal/registro.html'" class="accion" style="width:20 px;
77           ">Back</button>
78       </center>
79     </div>
80   </div>
81 </div>
82 </body>
83 </html>
84
85 <?php
86 exit; // Detenemos el flujo aquí, el formulario se genera y espera el envío.
87 }
88
89 // Procesamiento del formulario principal
90 if (isset($_POST['usuario']) && isset($_POST['name']) && isset($_POST['password']) && isset($_POST['email'])) {
91   $usuario = $_POST['usuario'];
92   $nombre = $_POST['name'];
93   $password = $_POST['password'];
94   $hashed_password = password_hash($password, PASSWORD_DEFAULT); // Encriptación segura
95   $email = $_POST['email'];
96   $tipo = $_POST['tipo'];
97
98   $conn = require("conexion.php");
99
100   $conn = require("conexion.php");
101
102   $sql = "INSERT INTO usuarios (usuario, password, nombre, email, tipo)
103     VALUES ('$usuario', '$hashed_password', '$nombre', '$email', '$tipo)";
104
105   if ($conn->query($sql) === TRUE) {
106     $mensaje = "Nuevo registro creado con éxito";
107   } else {
108     $mensaje = "Error: " . $conn->error;
109   }
110   $conn->close();
111 }
112

```

Tras el registro, el sistema muestra mensajes de éxito o error según el resultado de la operación, y la interfaz incluye una barra de navegación y un diseño responsivo para facilitar la interacción.

```

102 // Mostrar HTML nuevamente después de procesar
103 ?>
104 <!DOCTYPE html>
105 <html lang="en">
106 <head>
107 <meta charset="UTF-8">
108 <meta name="viewport" content="width=device-width, initial-scale=1.0">
109 <link rel="stylesheet" href="../styles/admin.css">
110 <link rel="stylesheet" href="../styles/navM.css">
111 <title>Registro</title>
112 </head>
113 <body>
114 <!-- Barra de navegación -->
115 <nav class="barra">
116 <ul class="topnav">
117 <li><a class="active" href="../paginaA.php">Inicio</a></li>
118 <li><a href="logout.php">Logout</a></li>
119 <li class="right"><a href="ayuda.php">Help</a></li>
120 </ul>
121 </nav>
122 <div class="contenido">
123 <div class="caja">
124 <div class="tipo">
125 <center>
126 <?php if (isset($mensaje)): ?>
127 <h2><?php echo $mensaje; ?></h2>
128 <button onclick="window.location.href='https://orangered-donkey-303339.hostingersite.com/proyectoFinal/registro.html'" class="accion">Back</button>
129 <?php else: ?>
130 <h1>Formulario de Registro</h1>
131 <form action="registrar.php" method="post">
132 <input type="hidden" name="tipo" value="<?php echo $role; ?>">
133 <?php if ($role == "2") { ?>
134 <label for="usuario">Usuario (Matricula):</label><br>
135 <input type="text" name="usuario" id="usuario" pattern="\d{6}" title="Debe ser una clave de 6 dígitos" required><br>
136 <?php } else { ?>
137 <label for="usuario">Nombre de usuario:</label><br>
138 <input type="text" name="usuario" id="usuario" required><br>
139 <?php } ?>
140
141 <label for="name">Nombre completo:</label><br>
142 <input type="text" name="name" id="name" required><br>
143
144 <label for="password">Contraseña:</label><br>
145 <input type="password" name="password" id="password" required><br>
146
147 <label for="email">Correo electrónico:</label><br>
148 <input type="email" name="email" id="email" required><br>
149
150 <button type="submit" class="accion">Registrar</button><br>
151 </form>
152 <?php endif; ?>
153 </center>
154 </div>
155 </div>
156 </div>
157 </body>
158 </html>

```

3.6 BajaUsuario.php

Inicio de sesión: Se utiliza `session_start()` para iniciar o continuar la sesión.

Control de acceso: Si no hay sesión iniciada o el rol del usuario no es permitido (código 0), se redirige a la página correspondiente.

```

<?php
session_start();

// Verifica si la sesión está iniciada
if (!isset($_SESSION['tipo'])) {
    // Si no está iniciada la sesión, redirige al login
    header('Location: https://orangered-donkey-303339.hostingersite.com/proyectoFinal/login.php');
    exit(); // Detiene la ejecución del script para evitar que se siga ejecutando el resto del código
} else {
    $role = $_SESSION['tipo']; // Obtén el rol del usuario desde la sesión
}

// Verifica si el rol no es 2 (no tiene permiso para acceder)
if ($role != '0') {
    header('Location: acceso_denegado.php'); // Redirige a una página de acceso denegado
    exit();
}

?>

```

Navegación: Proporciona enlaces a las páginas principales, incluyendo inicio, cierre de sesión y ayuda.

```
<!-- Barra de navegación -->
<nav class="barra">
  <ul class="topnav">
    <li><a class="active" href="../paginaA.php">Inicio</a></li>
    <li><a href="logout.php">Logout</a></li>
    <li class="right"><a href="ayuda.php">Help</a></li>
  </ul>
</nav>
```

Formulario: Permite al administrador ingresar un nombre de usuario para eliminar.

Validación: El campo de texto es obligatorio (required).

```
<div class="contenido">
  <div class="caja">
    <div class="tipo">
      <center>
        <h1>BAJA DE USUARIO</h1>
        <form action="bajaUsuario.php" method="post">
          <div class="li">
            <label for="">Usuario</label>
            <input type="text" name="usuario" class="text" required>
          </div>
          <input type="submit" value="Eliminar Usuario" class="accion">
        </form>
      </center>
    </div>
  </div>
</div>
```

1. Validación de Entrada: Se asegura de que el campo usuario no esté vacío antes de proceder.
2. Consulta: Comprueba si el usuario existe antes de intentar eliminarlo.
3. Eliminación: Realiza la operación DELETE en la base de datos si el usuario es válido.
4. Errores: Muestra mensajes de error si algo falla.

```

<?php
if(isset($_POST['usuario']) && !empty($_POST['usuario'])) {
    $conn = require("conexion.php");
    $usuario = $_POST['usuario'];

    // Verificar si el usuario existe
    $checkUser = "SELECT * FROM usuarios WHERE usuario = '$usuario'";
    $result = $conn->query($checkUser);

    if ($result->num_rows > 0) {
        // El usuario existe para proceder con la eliminación
        $sql = "DELETE FROM usuarios WHERE usuario = '$usuario'";

        if ($conn->query($sql) === TRUE) {
            echo "<h5>Usuario eliminado correctamente</h5>";
        } else {
            echo "<h5>Error al eliminar el usuario: " . $conn->error . "</h5>";
        }
    } else {
        // Usuario no encontrado
        echo "<h5>El usuario no existe.</h5>";
    }

    $conn->close();
} elseif(isset($_POST['usuario']) && empty($_POST['usuario'])) {
    echo "<h5>Por favor, ingresa un nombre de usuario.</h5>";
}
?>

```

5. Botón: Permite regresar a la pagina principal.

6. `<br><button onclick="window.location.href='https://orangered-donkey-303339.hostingersite.com/proyectoFinal/paginaA.php'" class="accion" style="width:20 px;">Back</button>`

3.2 Consulta.php

Este módulo permite consultar información específica de usuarios registrados en el sistema, solo accesible para administradores (rol 0). Se valida la sesión y el rol antes de cargar el formulario. Al ingresar el nombre de usuario, se realiza una consulta en la base de datos y se muestran los datos como usuario, nombre, email y tipo (Administrador, Profesor o Alumno). Incluye opciones para modificar al usuario o regresar al panel principal. La implementación utiliza una interfaz responsiva y la conexión a la base de datos es cerrada correctamente tras cada consulta.


```

18 <!DOCTYPE html>
19 <html lang="es">
20 <head>
21   <meta charset="UTF-8">
22   <meta name="viewport" content="width=device-width, initial-scale=1.0">
23   <title>Consulta de Usuario</title>
24   <link rel="stylesheet" href="../styles/admin.css">
25   <link rel="stylesheet" href="../styles/navM.css">
26 </head>
27 <body>
28   <!-- Barra de navegación -->
29   <nav class="barra">
30     <ul class="topnav">
31       <li><a class="active" href="../paginaA.php">Inicio</a></li>
32       <li><a href="logout.php">Logout</a></li>
33       <li class="right"><a href="ayuda.php">Help</a></li>
34     </ul>
35   </nav>
36
37   <div class="contenido">
38     <div class="caja">
39       <div class="tipo">
40         <center>
41           <h1>CONSULTA DE USUARIO</h1><br>
42           <form action="consulta.php" method="post">
43             <div class="li">
44               <b><label for="">Usuario</label></b>
45               <input type="text" name="usuario" class="text">
46             </div>
47             <br><input type="submit" value="Consultar Usuario" class="accion">
48           </form>
49
50         </center>
51       </div>
52     <div class="tipo">
53       <center>
54         <?php
55           if(isset($_POST['usuario'])){
56             $conn = require("conexion.php");
57             $usuario = $_POST['usuario'];
58             $sql = "SELECT * FROM usuarios WHERE usuario='".$usuario."'";
59             $result = $conn->query($sql);
60
61             $_SESSION['usuario']=$usuario;
62
63             if($result->num_rows > 0){
64
65               while($row = $result->fetch_assoc()){
66                 echo "<div class='li consulta'><label for=''><b>Usuario</b> </label><input type='text' name='name' class='text' value='".$row['usuario']."' readonly>
67                 </div>";
68                 echo "<div class='li consulta'><label for=''><b>Nombre</b> </label><input type='text' name='name' class='text' value='".$row['nombre']."' readonly>
69                 </div>";
70                 echo "<div class='li consulta'><label for=''><b>Email</b> </label><input type='text' name='email' class='text' value='".$row['email']."' readonly>
71                 </div>";
72
73                 switch($row['tipo']){
74                   case 0:
75                     $tipo = "Administrador";
76                     break;
77                   case 1:
78                     $tipo = "Profesor";
79                     break;
80                   case 2:
81                     $tipo = "Alumno";
82                     break;
83                 }
84
85                 echo "<div class='li consulta'><label for=''><b>Tipo</b> </label><input type='text' name='name' class='text' value='".$tipo."' readonly></div>";
86                 echo "<button class='accion'><a href='modificar.php' style='color: white'>Modificar Usuario</a></button>";
87               }
88             }
89             $conn->close();
90           }
91       >
92       <br><button onclick="window.location.href='https://orangered-donkey-303339.hostingersite.com/proyectoFinal/paginaA.php'" class="accion" style="width:20 px;">Back
93     </center>
94   </div>
95 </div>
96
97 </div>
98
99 </body>
100 </html>

```

3.3 Modificar.php

1. Inicio de Sesión y Verificación de Permisos

2. Formulario de Consulta de Usuario

```
<center>
<form action="modificar.php" method="post">
  <div class="li">
    <b><label for="">Usuario</label></b>
    <input type="text" name="usuario" class="text">
  </div>
  <br><input type="submit" value="Prepare para modificar" class="accion">
</form>
```

- Si el usuario tiene permisos, se presenta un formulario HTML en la interfaz donde el Administrador puede ingresar el nombre de usuario que desea consultar.
- Al enviar este formulario, el código captura el valor del campo usuario y realiza una consulta SQL a la base de datos para obtener los detalles del usuario correspondiente. 4 Muestra de Datos del Usuario

```
<form action="modificarUsuario.php" method="post">
  <?php
    if(isset($_POST['usuario'])){
      $conn = require("conexion.php");
      $usuario = $_POST['usuario'];
      $sql = "SELECT * FROM usuarios WHERE usuario='".$_usuario.'";
      $result = $conn->query($sql);

      $_SESSION['usuario']=$usuario;

      if($result->num_rows > 0){
        while($row = $result->fetch_assoc()){
          echo "<div class='li' consulta><label for=''><b> Usuario </b></label><input type='text' name='usuario' class='text' value='".$row['usuario']."'
            readonly</div>";
          echo "<div class='li' consulta><label for=''><b>Nombre</b></label><input type='text' name='name' class='text' value='".$row['nombre']."' ></div>";
          echo "<div class='li' consulta><label for=''><b>Email</b></label><input type='text' name='email' class='text' value='".$row['email']."'></div>";

          switch($row['tipo']){
            case 0:
              $tipo = "Administrador";
              break;
            case 1:
              $tipo = "Profesor";
              break;
            case 2:
              $tipo = "Alumno";
              break;
          }
          echo "<div class='li'>
            <label for=''><b>Tipo</b></label>
            <label for=''><b>Tipo</b></label>
            <select name='tipo' id='' value='".$tipo.">
              <option value='0'>Administrador</option>
              <option value='1'>Maestro</option>
              <option value='2'>Alumno</option>
            </select>
          </div>";
        }
      }
      echo "<br><input type='submit' value='Modificar Usuario' class='accion'>";
      $conn->close();
    }
  >
```

- El campo de tipo de usuario es desplegado como un menú de selección (<select>), permitiendo cambiar el rol del usuario entre las opciones Administrador, Profesor, y Alumno.
 - El valor del campo usuario se mantiene en solo lectura, evitando que el Administrador modifique este dato.
- #### 4. Envío de Datos para Modificación

- Después de que el Administrador haya revisado o modificado los datos del usuario, el formulario es enviado a otro script (modificarUsuario.php) para procesar la actualización en la base de datos.

3.4 modificarUsuario.php

```
<?php
if(isset($_POST['usuario']) && isset($_POST['name']) && isset($_POST['email']) && isset($_POST['tipo'])){
    $conn = require("conexion.php");
    $usuario = $_POST['usuario'];
    $name = $_POST['name'];
    $email = $_POST['email'];
    $tipo = $_POST['tipo'];

    if($tipo!=0 && $tipo!=1 && $tipo!=2){
        $sql = "UPDATE usuarios SET nombre='".$name."', email='".$email."' WHERE usuario='".$usuario."' ";
    }else{
        $sql = "UPDATE usuarios SET nombre='".$name."', email='".$email."', tipo='".$tipo."' WHERE usuario='".$usuario."' ";
    }
    $conn->query($sql);
    echo "el usuario se modifico";
    header("Location: https://orangered-donkey-303339.hostingersite.com/proyectoFinal/archivosPHP/modificar.php");
}
?>
```

- Validación de Datos: Se realiza una validación básica para el campo tipo. Si este no corresponde a los valores permitidos (0, 1, o 2), el tipo de usuario no es actualizado.
- Redirección y Mensaje de Confirmación: Aunque el mensaje de confirmación es mostrado con echo, es poco probable que sea visible debido a la redirección inmediata con header(). Considera usar una página intermedia o un sistema de notificaciones para mejorar la experiencia del usuario.

3.5 ConsultaPlan.php

1. Control de acceso: Garantiza que solo los usuarios autorizados (rol 0) puedan acceder al sistema.
2. Redirecciones: Redirige a las páginas de inicio de sesión o de acceso denegado según sea necesario.

```
<?php
session_start();

// Verifica si la sesión está iniciada
if (!isset($_SESSION['tipo'])) {
    header('Location: https://orangered-donkey-303339.hostingersite.com/proyectoFinal/login.php');
    exit();
} else {
    $role = $_SESSION['tipo'];
}

// Verifica si el rol no es 0 (no tiene permiso para acceder)
if ($role != '0') {
    header('Location: acceso_denegado.php');
    exit();
}
?>
```

3. Enlaces principales: Incluye opciones para regresar al inicio, cerrar sesión y acceder a la página de ayuda.

```
<body>
  <nav class="barra">
    <ul class="topnav">
      <li><a class="active" href="../paginaA.php">Inicio</a></li>
      <li><a href="logout.php">Logout</a></li>
      <li class="right"><a href="ayuda.php">Help</a></li>
    </ul>
  </nav>
```

1. Formulario: Permite al usuario ingresar un identificador de plan para realizar la búsqueda.
2. Validación: El campo es obligatorio (required).

```
<div class="contenido">
  <div class="caja">
    <div class="tipo">
      <h1>CONSULTA DE PLAN DE ESTUDIOS</h1>
      <form action="consultaPlan.php" method="post">
        <div class="li">
          <label for="plan">PLAN DE ESTUDIOS:</label>
          <input type="text" name="plan" id="plan" class="text" value="" required>
        </div>
        <input type="submit" value="Consultar" class="accion">
      </form>
      <br><button onclick="window.location.href='https://orangered-donkey-303339.hostingersite.com/proyectoFinal/paginaA.php'" class="accion" style="width:20 px;">Back</button>
    </div>
  </div>
```

1. Validación de Entrada: Utiliza htmlspecialchars para prevenir inyecciones de código.
2. Consultas Preparadas: Garantiza la seguridad contra inyecciones SQL.
3. Impresión de Resultados: Combina los resultados de dos tablas diferentes y los muestra en formato de tabla.

```

<?php
if ($_SERVER['REQUEST_METHOD'] === 'POST' && isset($_POST['plan']) && !empty($_POST['plan'])) {
    $plan = htmlspecialchars($_POST['plan']); // Evita inyecciones de código.

    // Conexión a la base de datos.
    $conn = require("conexion.php");

    // Consulta en la primera tabla.
    $sql1 = "SELECT * FROM plan_estudios WHERE plan LIKE ? ORDER BY semestre";
    $stmt1 = $conn->prepare($sql1);
    $searchTerm = $plan . '%';
    $stmt1->bind_param("s", $searchTerm);
    $stmt1->execute();
    $result1 = $stmt1->get_result();

    // Consulta en la segunda tabla.
    $sql2 = "SELECT * FROM plan_estudios_item WHERE plan LIKE ? ORDER BY semestre";
    $stmt2 = $conn->prepare($sql2);
    $stmt2->bind_param("s", $searchTerm);
    $stmt2->execute();
    $result2 = $stmt2->get_result();

    // Verifica si hay resultados en alguna tabla.
    if ($result1->num_rows > 0 || $result2->num_rows > 0) {
        echo "<table>";
        echo "<tr>";
            <th>CLAVE</th>
            <th>SEMESTRE</th>
            <th>NOMBRE DE LA MATERIA</th>
            <th>CREDITOS</th>
            <th>HORAS</th>
            <th>PLAN</th>
        </tr>";

        // Imprime los resultados de la primera tabla.
        while ($row = $result1->fetch_assoc()) {
            echo "<tr>";
                <td> . $row['clave'] . "</td>
                <td> . $row['semestre'] . "</td>
                <td> . $row['nombre'] . "</td>
                <td> . $row['creditos'] . "</td>
                <td> . $row['horas'] . "</td>
                <td> . $row['plan'] . "</td>
            </tr>";
        }

        // Imprime los resultados de la segunda tabla.
        while ($row = $result2->fetch_assoc()) {
            echo "<tr>";
                <td> . $row['clave'] . "</td>
                <td> . $row['semestre'] . "</td>
                <td> . $row['nombre'] . "</td>
                <td> . $row['creditos'] . "</td>
                <td> . $row['horas'] . "</td>
                <td> . $row['plan'] . "</td>
            </tr>";
        }
        echo "</table>";
    } else {
        echo "<p>No se encontraron resultados para el plan ingresado en ninguna tabla.</p>";
    }

    $stmt1->close();
    $stmt2->close();
    $conn->close();
} else {
    if ($_SERVER['REQUEST_METHOD'] === 'POST') {
        echo "<p>Por favor, ingresa un plan de estudios.</p>";
    }
}
?>

```

3.6 Importación de Kardex.

```
// Verificar si el archivo es Excel o CSV
if ($tipo == 'text/csv') {
    $lineas = file($archTemp);
} elseif ($tipo == 'application/vnd.openxmlformats-officedocument.spreadsheetml.sheet' || $tipo == 'application/vnd.ms-excel') {
    // Si es Excel, se usa un spreadsheet para leerlo
    try {
        $spreadsheet = \PhpOffice\PhpSpreadsheet\IOFactory::load($archTemp);
        $sheet = $spreadsheet->getActiveSheet();

        // Leer datos de celdas específicas
        $matricula = $sheet->getCell('B3')->getFormattedValue();
        $carrera = $sheet->getCell('C3')->getFormattedValue();
        $matricula = substr($matricula, 3);

        // Verificar si la matrícula ya existe en la base de datos
        $matriculaEsc = $conn->real_escape_string($matricula);
        $query = "SELECT COUNT(*) AS total FROM kardex WHERE matricula = '$matriculaEsc'";
        $result = $conn->query($query);
        $row = $result->fetch_assoc();

        if ($row['total'] > 0) {
            $mensaje = "La matrícula ya existe en la base de datos. No se importará el archivo.";
        } else {
            $nombre = $sheet->getCell('E3')->getFormattedValue();
            $apellido = $sheet->getCell('F3')->getFormattedValue();
            $matricula = $sheet->getCell('G3')->getFormattedValue();
            $carrera = $sheet->getCell('H3')->getFormattedValue();
            $periodo = $sheet->getCell('I3')->getFormattedValue();
            $cfo = $sheet->getCell('J3')->getFormattedValue();
            $seccion = $sheet->getCell('K3')->getFormattedValue();

            $carrera = $sheet->getCell('L3')->getFormattedValue();
            $matricula = $sheet->getCell('M3')->getFormattedValue();
            $carrera = $sheet->getCell('N3')->getFormattedValue();

            $fila = 5;

            while ($sheet->getCell('N' . $fila)->getFormattedValue() != '') {
                $sem = (int)$sheet->getCell('B' . $fila)->getFormattedValue();
                $materia = $sheet->getCell('C' . $fila)->getFormattedValue();
                $matricula = $sheet->getCell('D' . $fila)->getFormattedValue();
                $seccion = $sheet->getCell('E' . $fila)->getFormattedValue();
                $seccion = $sheet->getCell('F' . $fila)->getFormattedValue();
                $periodo = $sheet->getCell('G' . $fila)->getFormattedValue();
                $cfo = (float)$sheet->getCell('H' . $fila)->getFormattedValue();
                $ext = (float)$sheet->getCell('I' . $fila)->getFormattedValue();
                $reg = (float)$sheet->getCell('J' . $fila)->getFormattedValue();
                $cf = (float)$sheet->getCell('K' . $fila)->getFormattedValue();
                $creditos = (int)$sheet->getCell('M' . $fila)->getFormattedValue();
                $status = $sheet->getCell('N' . $fila)->getFormattedValue();
            }
        }
    } catch (Exception $e) {
        // Manejar errores de lectura de Excel
    }
}
```

Imagen 1. Importación de Kardex.

Este código PHP permite cargar un archivo Excel o CSV, procesar sus datos y almacenarlos en una base de datos MySQL, específicamente en la tabla kardex. Utiliza la librería PhpSpreadsheet para leer los datos del archivo y comienza verificando si el archivo es válido. Extrae información específica como matrícula, nombre, carrera y detalles académicos desde las celdas del archivo, luego comprueba si la matrícula ya existe en la base de datos para evitar duplicados. Si no existe, inserta los datos fila por fila. Además, genera un mensaje de éxito o error según el resultado del proceso, y muestra una interfaz HTML para notificar al usuario el estado de la importación.

```

<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Administrador</title>
  <link rel="stylesheet" href="../styles/importStyle.css">
  <link rel="stylesheet" href="../styles/navM.css">
</head>
<body>
  <!-- Barra de navegación -->
  <nav class="barra">
    <ul class="topnav">
      <li><a href="../paginaA.php">Inicio</a></li>
      <li><a href="/archivosPHP/logout.php">Logout</a></li>
      <li class="right"><a href="../archivosPHP/ayuda.php">Help</a></li>
    </ul>
  </nav>

  <header>¡Bienvenido a la Página de Importación de Kardex!</header>

  <div class="card-container">
    <div class="card">
      <form method="post" action="importarKardex.php" enctype="multipart/form-data" >
        <h2>Selecciona el archivo Excel</h2>
        <input type="file" name="excel">
        <button type="submit">Importar Datos</button>
      </form>
    </div>
  </div>
</body>
</html>

```

Imagen 2. Kardex HTML.

Este código HTML define una página web para la importación de datos, con una barra de navegación que incluye enlaces a la página de inicio, a cerrar sesión y a la página de ayuda. La sección principal presenta un encabezado de bienvenida y un formulario para subir un archivo Excel con los datos del Kardex. El formulario, con el método post y el atributo `enctype="multipart/form-data"`, permite al usuario seleccionar un archivo Excel y enviarlo a un archivo PHP (`importarKardex.php`) para su procesamiento. Además, se incluyen dos hojas de estilo externas (`importStyle.css` y `navM.css`) para los estilos específicos de la página y la barra de navegación.

3.7 Importación de plan de estudios.


```
// Verificar si el archivo es válido
if ($tipo == 'text/csv') {
    $lineas = file($archTemp);
} elseif ($tipo == 'application/vnd.openxmlformats-officedocument.spreadsheetml.sheet' || $tipo == 'application/vnd.ms-excel') {
    try {
        $spreadsheet = \PhpOffice\PhpSpreadsheet\IOFactory::load($archTemp);
        $sheet = $spreadsheet->getActiveSheet();

        // Leer el código del plan desde la columna correspondiente
        $fila = 7; // Fila inicial según el formato del archivo
        $plan = $sheet->getCell('C' . $fila)->getFormattedValue();
        $planEsc = $conn->real_escape_string($plan);

        // Verificar si el plan ya existe en la base de datos
        if ($tipo_plan == 0) {
            $query = "SELECT COUNT(*) AS total FROM plan_estudios WHERE plan = '$planEsc'";
            $tabla = "plan_estudios";
        } else {
            $query = "SELECT COUNT(*) AS total FROM plan_estudios_item WHERE plan = '$planEsc'";
            $tabla = "plan_estudios_item";
        }

        $result = $conn->query($query);
        $row = $result->fetch_assoc();

        if ($row['total'] > 0) {
            $mensaje = "El plan '$plan' ya existe en la tabla $tabla. No se insertarán datos.";
        } else {
            // Si el plan no existe, proceder a insertar los datos
            while ($sheet->getCell('H' . $fila)->getFormattedValue() != '') {
                $claveM = $sheet->getCell('D' . $fila)->getFormattedValue();
                $nombre = $sheet->getCell('E' . $fila)->getFormattedValue();
                $nivelClase = (int)$sheet->getCell('F' . $fila)->getFormattedValue();
                $creditos = (int)$sheet->getCell('G' . $fila)->getFormattedValue();
                $horasSemana = (int)$sheet->getCell('H' . $fila)->getFormattedValue() * 16;

                $claveMEsc = $conn->real_escape_string($claveM);
                $nombreEsc = $conn->real_escape_string($nombre);

                $sql = "INSERT INTO $tabla (clave, semestre, nombre, creditos, horas, plan)
                    VALUES ('$claveMEsc', '$nivelClase', '$nombreEsc', '$creditos', '$horasSemana', '$planEsc')";

                if ($conn->query($sql) === FALSE) {
                    $mensaje = "Error al insertar datos en la tabla $tabla: " . $conn->error;
                    break;
                }

                $fila++;
            }

            if ($mensaje === "") {
                $mensaje = "Importación exitosa. Los datos del plan '$plan' se han cargado correctamente.";
            }
        }
    }
}
```

Imagen 3. Importación de plan de estudios

Este código PHP maneja la carga e importación de un archivo Excel o CSV, el cual contiene datos de planes de estudios y los inserta en una base de datos MySQL. Primero, se recibe el archivo desde un formulario y se verifica que sea del tipo correcto (CSV o Excel). Luego, se lee el archivo con la librería PhpSpreadsheet y se extrae el código del plan desde una celda específica en la fila 7 del archivo. El código verifica si el plan ya existe en una de dos tablas posibles (plan_estudios o plan_estudios_item) en la base de datos, dependiendo del valor recibido de un formulario. Si el plan ya existe, se muestra un mensaje indicando que no se insertarán datos. Si el plan no existe, el código recorre las filas del archivo e inserta los datos correspondientes a cada materia (clave, nombre, semestre, créditos, horas) en la tabla elegida. Si ocurre un error en la inserción, se muestra un mensaje de error. Finalmente, se muestra un mensaje en la interfaz HTML notificando al usuario si la importación fue exitosa o si ocurrió un error, junto con un enlace para regresar al formulario de carga.

```

<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Administrador</title>
  <link rel="stylesheet" href="../styles/importStyle.css">
  <link rel="stylesheet" href="../styles/navM.css">
</head>
<body>
  <!-- Barra de navegación -->
  <nav class="barra">
    <ul class="topnav">
      <li><a href="../paginaA.php">Inicio</a></li>
      <li><a href="/archivosPHP/logout.php">Logout</a></li>
      <li class="right"><a href="/archivosPHP/ayuda.php">Help</a></li>
    </ul>
  </nav>

  <header>¡Bienvenido a la Página de Alta de Plan de Estudios!</header>

  <div class="card-container">
    <div class="card">
      <form method="post" action="importarPlan.php" enctype="multipart/form-data">
        <h2>Selecciona el archivo Excel</h2>
        <input type="file" name="excel" required>
        <h2>Selecciona el tipo de plan</h2>
        <select name="tipo_plan" required>
          <option value="" disabled selected>Selecciona un tipo de plan</option>
          <option value="0">ITI</option>
          <option value="1">ITEM</option>
        </select>
        <button type="submit">Importar Plan</button>
      </form>
    </div>
  </div>
</body>
</html>

```

Imagen

4. Plan de estudios HTML.

Este código HTML crea una página web con un formulario para importar un archivo Excel que contiene los datos del plan de estudios. La página tiene una barra de navegación con enlaces a la página de inicio, de cierre de sesión y de ayuda. El formulario permite al usuario seleccionar un archivo Excel mediante un campo de entrada (<input type="file">) y escoger un tipo de plan de estudios a través de un menú desplegable (<select>), que ofrece dos opciones: "ITI" y "ITEM". El formulario se envía al archivo importarPlan.php mediante el método POST para procesar los datos del archivo cargado. Además, la página incluye dos hojas de estilo externas para el diseño (importStyle.css y navM.css).

```

margin: 50px auto;
gap: 20px;
padding: 0 20px;
}
/* Cards */
.card {
background-color: rgba(255, 250, 250, 0.785);
width: 350px;
padding: 20px;
border-radius: 10px;
box-shadow: 0 8px 15px rgba(0, 0, 0, 0.1);
text-align: center;
transition: transform 0.3s ease, box-shadow 0.3s ease;
}

.card:hover {
transform: translateY(-10px);
box-shadow: 0 12px 20px rgba(0, 0, 0, 0.2);
}

.card input {
font-size: 14px;
margin-bottom: 10px;
}

.card h2 {
font-size: 24px;
color: #333;
margin-bottom: 15px;
}

.card p {
font-size: 16px;
color: #555;
margin-bottom: 20px;
}

.card a, button {
display: inline-block;
text-decoration: none;
color: white;
background-color: #e77817;
padding: 10px 20px;
border-radius: 5px;
font-size: 18px;
transition: background-color 0.3s ease;
}

.card a:hover, button:hover {
background-color: #cf6407;
}

```

Imagen 5. CSS de importación.

Este código CSS define un diseño limpio y moderno para una página web, utilizando un fondo de gradiente azul para el cuerpo y una fuente 'Arial' para todo el texto. El encabezado está centrado y en mayúsculas, con un espaciado entre letras para destacar su presencia. Las tarjetas están organizadas dentro de un contenedor flexible que las alinea y espacia uniformemente. Cada tarjeta tiene un fondo semitransparente, bordes redondeados y una sombra que aumenta al pasar el cursor, lo que da un efecto de elevación. Los títulos y párrafos dentro de las tarjetas están claramente definidos para una mejor legibilidad. Los botones y enlaces tienen un fondo naranja y texto blanco, con un efecto visual que oscurece el fondo al pasar el cursor, mejorando la interactividad. En general, el diseño está pensado para ser visualmente atractivo y fácil de usar, con una interfaz moderna y fluida.

4. AYUDA

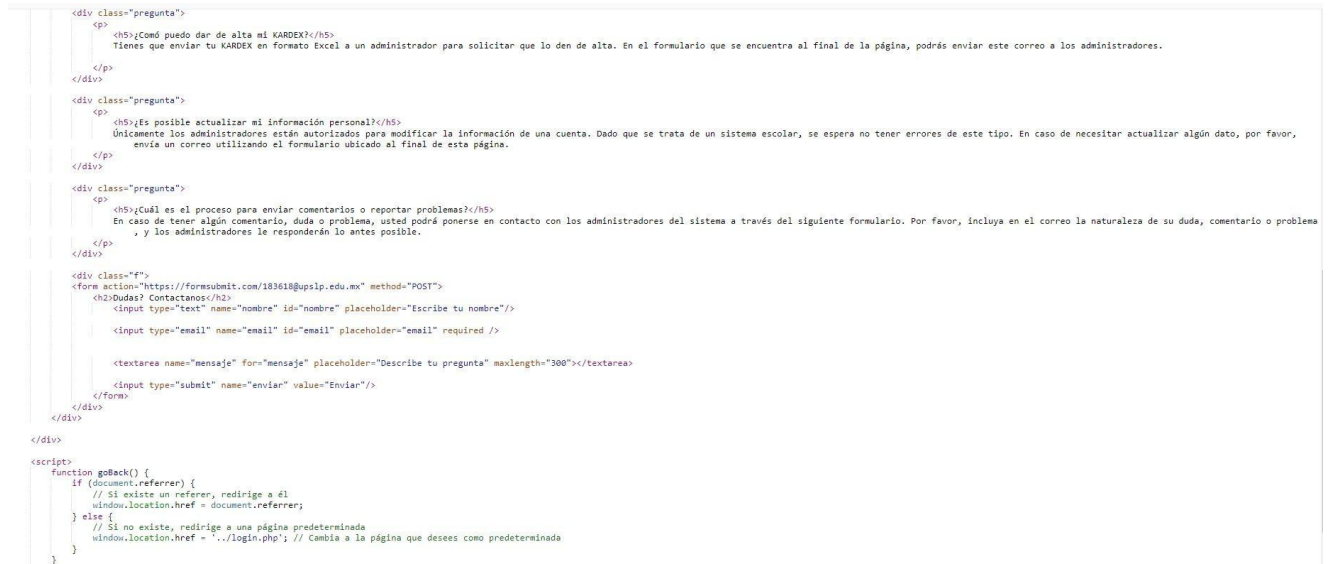


Imagen 6. Sección de ayuda.

Este código HTML crea una página web de ayuda para los usuarios de un sistema, con varias secciones explicativas sobre cómo usar la plataforma. La página incluye una barra de navegación con enlaces para volver a la página principal, cerrar sesión y acceder a la sección de ayuda. En la sección principal de la página, se detallan varias preguntas frecuentes (FAQ) sobre el sistema, como cómo registrarse, las funciones del sistema, cómo dar de alta el KARDEX, la posibilidad de actualizar información personal y cómo enviar comentarios o reportar problemas. Al final de la página, hay un formulario de contacto que permite a los usuarios enviar dudas o comentarios a los administradores. Este formulario utiliza el servicio de FormSubmit para enviar los mensajes a un correo electrónico. Además, el código incluye enlaces a varias hojas de estilo para el diseño de la página. También se incluye un script para permitir a los usuarios regresar a la página anterior si es posible, o redirigirlos a una página predeterminada si no lo es.

```

.f form {
  background-color: rgba(255, 255, 255, 0.15);
  padding: 20px 30px;
  border-radius: 10px;
  box-shadow: 0 8px 15px rgba(0, 0, 0, 0.2);
  width: 80%;
  max-width: 500px;
  color: white;
  text-align: center;
}

.f form h2 {
  font-size: 22px;
  margin-bottom: 15px;
  color: #ffffff;
}

.f form input, .f form textarea {
  width: 100%;
  padding: 10px;
  margin: 10px 0;
  border: none;
  border-radius: 5px;
  font-size: 14px;
}

.f form input[type="submit"] {
  background-color: #e77817;
  color: white;
  border: none;
  font-size: 16px;
  padding: 10px 15px;
  cursor: pointer;
  transition: background-color 0.3s ease;
}

.f form input[type="submit"]:hover {
  background-color: #cf6407;
}

/* Ajustes responsivos */
@media screen and (max-width: 600px) {
  .pregunta {
    padding: 10px;
  }

  .f form {
    width: 90%;
  }
}

```

Imagen 7. CSS de ayuda.

Este código CSS define un diseño limpio y moderno para una página web, con un fondo de gradiente azul y texto blanco en todo el cuerpo de la página. La fuente utilizada es 'Arial', con un espaciado de líneas de 1.6 para una mejor legibilidad. El encabezado está centrado, en mayúsculas, con un tamaño de fuente grande y un espaciado entre letras para destacar. El contenido se organiza dentro de un contenedor con un ancho máximo de 1200px, centrado y con un poco de relleno. Las preguntas están destacadas en tarjetas con un fondo semitransparente blanco y una sombra sutil, con el título en un color más claro y el texto en gris. El formulario de contacto está centrado, con un fondo oscuro semitransparente y bordes redondeados, acompañado de un botón naranja que cambia de color cuando se pasa el cursor. Además, se implementa un diseño responsivo para pantallas más pequeñas, ajustando el tamaño de los elementos y las tarjetas de preguntas.

5. ESTADISTICA CONTROL.PHP

1. Inicio de sesión: Utiliza `session_start()` para iniciar o continuar la sesión.
2. Validación de roles: Identifica el tipo de usuario a través de `$_SESSION['tipo']` y redirige a diferentes URL según su rol.
3. Protección de acceso: Redirige a una página de acceso denegado para usuarios con roles no autorizados.
4. Inclusión del archivo: Usa un archivo externo `conexion.php` para conectar a la base de datos.
5. Codificación: Configura el conjunto de caracteres a UTF-8 para manejar adecuadamente los datos en español.

```
<?php
session_start();

// Verifica si la sesión está iniciada
if (!isset($_SESSION['tipo'])) {
    header('Location: https://orangered-donkey-303339.hostingersite.com/proyectoFinal/login.php');
    exit();
} else {
    $role = $_SESSION['tipo'];
}

// Determina la URL de inicio según el rol
if ($role == '0') {
    $inicioUrl = '../paginaA.php';
} else {
    if ($role == '1') {
        $inicioUrl = '../paginaM.html';
    } else {
        header('Location: https://orangered-donkey-303339.hostingersite.com/proyectoFinal/archivosPHP/acceso_denegado.php');
        exit();
    }
}

// Conexión a la base de datos
$conn = require("conexion.php");
$conn->set_charset("utf8");

?>
```

Navegación

```
<!-- Barra de navegación -->
<nav class="barra">
    <ul class="topnav">
        <li><a href="<?php echo $inicioUrl; ?>">Inicio</a></li>
        <li><a href="../archivosPHP/logout.php">Logout</a></li>
        <li class="right"><a href="../archivosPHP/ayuda.php">Help</a></li>
    </ul>
</nav>
```

Explicación

Proporciona enlaces para navegar al inicio, cerrar sesión y acceder a la ayuda. La URL de "Inicio" se ajusta según el rol del usuario.

1. Formulario: Cada carrera tiene su propio formulario que envía a una solicitud POST al script `estadisticaPDF.php`.
2. Campo oculto: Usa `input type="hidden"` para pasar el identificador de la carrera al script.

3. Botón: Diseñado con estilos CSS, el botón envía la solicitud al hacer clic.

```
<div class="card-container">

  <!-- Botón ITI -->
  <div class="card">
    <h2>ITI</h2>
    <form action="../../archivosPHP/estadisticaPDF.php" method="POST" target="_blank">
      <input type="hidden" name="carrera" value="ITI01">
      <button type="submit" class="boton-naranja">ALUMNOS ITI</button>
    </form>
  </div>

  <!-- Botón ITEM -->
  <div class="card">
    <h2>ITEM</h2>
    <form action="../../archivosPHP/estadisticaPDF.php" method="POST" target="_blank">
      <input type="hidden" name="carrera" value="ITEM02">
      <button type="submit" class="boton-naranja">ALUMNOS ITEM</button>
    </form>
  </div>

</div>
```

6. PDF ESTADÍSTICAS ESTUDIANTES.

6.1 PHP

```
estadisticaPDF.php
archivosPHP > estadisticaPDF.php
1 <?php
2
3 require('fpdf183/fpdf.php');
4
5 $conn=require("conexion.php");
6 $conn->set_charset("utf8");
7 session_start();
8
9 if ($_SERVER['REQUEST_METHOD'] === 'POST') {
10     if ( $_POST['carrera'] == 'ITI01' ) {
11         $planEstudios = "plan_estudios";
12         $carrera= "ITI01";
13     }else if(isset($_POST['carrera']) == 'ITEM02' ){
14         $planEstudios = "plan_estudios_item";
15         $carrera= "ITEM02";
16     }
17 }
18
19
20 function normalizarTexto($texto) {
21     $texto = trim($texto); // Elimina espacios adicionales al inicio y final
22     $texto = strtolower($texto); // Convierte todo a minúsculas
23     $texto = str_replace([':', '-', '_'], '', $texto); // Elimina caracteres especiales
24     $texto = preg_replace('/\s+/',' ', $texto); // Reduce múltiples espacios a uno solo
25     return $texto;
26 }
27
28 $materiasErroneas = [
29     "CNG I Desarrollo del Pensamiento Crítico",
30     "Curso del Núcleo Optativo I",
31     "Curso del Núcleo Optativo II",
32     "Curso del Núcleo Optativo III",
33     "Curso del Núcleo Optativo IV",
34     "Curso del Núcleo Optativo V"
35 ];
36
37 foreach ($materiasErroneas as $key => $materia) {
```

- fpdf183/fpdf.php: Incluye la biblioteca FPDF para la generación de documentos PDF.
- conexion.php: Realiza la conexión con la base de datos, centralizada en un archivo separado para facilitar la reutilización y el mantenimiento.

- `set_charset("utf8")`: Configura la conexión para manejar caracteres especiales como tildes y eñes.
- `session_start()`: Habilita el uso de sesiones para almacenar datos temporales a lo largo del script.
- Validación del Método HTTP: Se asegura de que el código solo se ejecute si el método de la solicitud es POST.
- Selección del Plan de Estudios:
 - Verifica el valor enviado desde el formulario (`$_POST['carrera']`).
 - Si la carrera es ITIO1, asigna la tabla correspondiente al plan de estudios.
 - Si es ITEM02, asigna la tabla alternativa.
- Normalización de textos para comparaciones `function normalizarTexto($texto)`
 - `trim`: Elimina espacios en blanco al inicio y al final del texto.
 - `strtolower`: Convierte el texto a minúsculas.
 - `str_replace`: Remueve caracteres específicos como `:`, `-` y `_`.
 - `preg_replace`: Sustituye múltiples espacios consecutivos por un único espacio.
- Listado de Materias Excluidas. Representa un conjunto de materias que se sustituirán por su correspondiente en el Kardex de cada alumno

```

estadisticaPDF.php
archivosPHP > estadisticaPDF.php
35     };
36
37     foreach ($materiasErroneas as $key => $materia) {
38         $materiasErroneas[$key] = normalizarTexto($materia); // Actualiza el valor del arreglo
39     }
40
41
42     $nombreAlumno = array();
43     $creditos = array();
44
45     $sql = "SELECT DISTINCT nombre FROM kardex WHERE carrera = '$carrera' ORDER BY nombre";
46     $result = $conn->query($sql);
47     if($result->num_rows > 0){
48         while( $row = $result->fetch_assoc()){
49             $nombreAlumno[] = $row['nombre'];
50         }
51     }
52
53
54     $sql = "SELECT REPLACE(nombre, ':', '') AS nombre, creditos FROM $planEstudios";
55     $result = $conn->query($sql);
56     if($result->num_rows > 0){
57         while( $row = $result->fetch_assoc()){
58             $creditos[ $row['nombre'] ] = $row['creditos'];
59         }
60     }
61
62     $alumnosConCreditos = array(); // Arreglo asociativo para almacenar los datos
63     $creditosFaltantes = array();
64     $creditosAprobados = array();
65
66     foreach ($nombreAlumno as $nombre) {
67         //Inicializa el array para el estudiante actual
68         $alumnosConCreditos[$nombre] = [
69             'creditos' => 0,
70             'porcentaje' => 0,
71             'largo' => 0,

```

- Normalización de Materias Erróneas. Normaliza los nombres de las materias marcadas como "erróneas" (o excluidas) para garantizar que puedan compararse de manera uniforme con los nombres de materias en otros conjuntos de datos.
 - Método: Utiliza la función `normalizarTexto` previamente definida para modificar cada elemento del arreglo `$materiasErroneas`.
- Obtención de Nombres de Alumnos
 - Consulta SQL: Recupera nombres únicos (`DISTINCT nombre`) de estudiantes inscritos en la carrera seleccionada.
 - Resultado: Si hay datos, los nombres se almacenan en el arreglo `$nombreAlumno`. Los nombres se ordenan alfabéticamente por `ORDER BY nombre`.
- Créditos del Plan de Estudios
 - Consulta SQL: Recupera nombres de materias y créditos asociados del plan de estudios correspondiente.
 - Transformación: Usa `REPLACE` para eliminar los dos puntos (:) del nombre de las materias antes de almacenarlas. Los resultados se almacenan en el arreglo asociativo `$creditos`, donde la clave es el nombre de la materia y el valor, los créditos.
- Estructura Inicial de Datos por Alumno
 - `creditos`: Total de créditos obtenidos.
 - `porcentaje`: Porcentaje de avance en los créditos totales.
 - `largo`: Longitud gráfica de la barra de progreso.
 - `restante`: Espacio restante en la barra de progreso.
 - `materiasAprobadas`: Lista de materias aprobadas por el estudiante.
 - `materiasFaltantes`: Lista de materias que el estudiante aún debe cursar.

```

estadísticaPDF.php
archivosPHP > estadísticaPDF.php
66 foreach ($nombreAlumno as $nombre) {
67     $alumnosConCreditos[$nombre] = [
68         'total' => 0,
69         'restante' => 0,
70         'materiasAprobadas' => [],
71         'materiasFaltantes' => []
72     ];
73
74     // Primera consulta: Créditos obtenidos por el estudiante
75     $sql = "SELECT SUM(creditos) AS total FROM kardex WHERE nombre = ?";
76     $stmt = $conn->prepare($sql);
77     $stmt->bind_param("s", $nombre);
78     $stmt->execute();
79     $result = $stmt->get_result();
80
81     if ($result->num_rows > 0) {
82         $row = $result->fetch_assoc();
83         $creditosObtenidos = $row['total'] ?? 0; // Manejar valores nulos
84         $porcentaje = round(($creditosObtenidos * 100) / 407); // Calcula el porcentaje
85         $largo = ($porcentaje * 180) / 100; // Longitud de la barra
86         $restante = 180 - $largo; // Diferencia restante
87
88         // Actualiza los datos del estudiante
89         $alumnosConCreditos[$nombre]['creditos'] = $creditosObtenidos;
90         $alumnosConCreditos[$nombre]['porcentaje'] = $porcentaje;
91         $alumnosConCreditos[$nombre]['largo'] = $largo;
92         $alumnosConCreditos[$nombre]['restante'] = $restante;
93
94         // Segunda consulta: Materias aprobadas (créditos > 0)
95         $sql = "SELECT REPLACE(materia, ':', ' ') AS materia, creditos FROM kardex WHERE creditos != 0 AND nombre = ?";
96         $stmt = $conn->prepare($sql);
97         $stmt->bind_param("s", $nombre);
98         $stmt->execute();
99         $result = $stmt->get_result();
100
101         $materiasAprobadas = [];
102     }
103 }

```

- Consultar Créditos Obtenidos por el Estudiante
 - Realiza una consulta a la tabla kardex para sumar todos los créditos que el estudiante ha obtenido.
 - Calcula el porcentaje de créditos obtenidos sobre un total fijo de 407 créditos.
 - Calcula dimensiones gráficas (\$largo y \$restante) para mostrar el avance en una barra de progreso.
 - SUM(creditos): Suma los créditos para un estudiante específico. round: Redondea el porcentaje a un número entero.
 - Se maneja el caso en que no haya créditos registrados utilizando ?? 0 para evitar errores.
- Actualización del Arreglo \$alumnosConCreditos
 - Almacena los resultados calculados (créditos, porcentaje y dimensiones gráficas) en un arreglo asociativo bajo el nombre del estudiante.
- Consultar Materias Aprobadas
 - Realiza una consulta para recuperar todas las materias que el estudiante ha aprobado (es decir, aquellas con créditos mayores a cero).
 - Los nombres de las materias se procesan para eliminar caracteres especiales como :.
 - REPLACE(materia, ':', ' '): Limpia los nombres de las materias antes de procesarlos.

- Arreglo \$materiasAprobadas: Utiliza los nombres de las materias como claves y sus créditos como valores para facilitar búsquedas rápidas.
- Arreglo

\$alumnosConCreditos: También almacena una lista detallada de las materias aprobadas por el estudiante, con su nombre y sus créditos.

```

estadísticaPDF.php
archivosPHP > estadísticaPDF.php
66  foreach ($nombreAlumno as $nombre) {
84  if ($result->num_rows > 0) {
103     $result = $stmt->get_result();
104
105     $materiasAprobadas = [];
106
107     if ($result->num_rows > 0) {
108         while ($row = $result->fetch_assoc()) {
109             $materiasAprobadas[$row['materia']] = $row['creditos']; // Clave: materia, Valor: créditos
110             $alumnosConCreditos[$nombre]['materiasAprobadas'][] = [
111                 'materia' => $row['materia'],
112                 'creditos' => $row['creditos']
113             ];
114         }
115     }
116
117     // Comparar con el plan de estudios ($creditos) para determinar materias faltantes
118     foreach ($creditos as $materia => $totalCreditos) {
119         // Normaliza el nombre de la materia en el plan de estudios
120         $materiaNormalizada = normalizarTexto($materia);
121
122         // Verifica si está aprobada utilizando el nombre normalizado
123         $materiaAprobadaNormalizada = array_map('normalizarTexto', array_keys($materiasAprobadas));
124
125         if (!in_array($materiaNormalizada, $materiaAprobadaNormalizada)) {
126             // Materia faltante, agregar al arreglo
127             if (!in_array($materiaNormalizada, $materiasErroneas)){
128                 $alumnosConCreditos[$nombre]['materiasFaltantes'][] = [
129                     'materia' => $materia,
130                     'creditos' => $totalCreditos
131                 ];
132             }
133         }
134     }
135 }
136 }
137 }

```

- Determinar Materias Faltantes
 - Compara la lista completa de materias del plan de estudios (\$creditos) con las materias aprobadas (\$materiasAprobadas) para identificar cuáles faltan.

Si una materia no está en la lista de aprobadas y tampoco está en la lista de materias erróneas (\$materiasErroneas), se considera como faltante.

- Normalización de texto:
 - Los nombres de las materias se pasan por la función normalizarTexto para eliminar inconsistencias como espacios adicionales, diferencias en mayúsculas/minúsculas y caracteres no alfanuméricos.
- in_array:
 - Compara las materias del plan de estudios con las aprobadas de forma eficiente.
- Lista de materias faltantes:
 - Cada materia faltante se almacena en el arreglo \$alumnosConCreditos con su nombre y créditos correspondientes.

```

estadísticaPDF.php X
archivosPHP > estadísticaPDF.php
138
139
140 class PDF extends FPDF{
141 // Cabecera de página
142 function Header()
143 {
144 // Logo
145 $this->Image('../images/UPSLPlogo.png',10,8,33);
146 // Arial bold 15
147 $this->SetFont('Arial','B',15);
148 // Movernos a la derecha
149 $this->Cell(50);
150 // Título
151 $this->Cell(100,10,'Reporte de avance',0,0,'C');
152 // Salto de línea
153 $this->Ln(20);
154 }
155 function ChapterTitle($num, $label)
156 {
157 // Arial 12
158 $this->SetFont('Arial','',12);
159 // Color de fondo
160 $this->SetFillColor(200,220,255);
161 // Título
162 $this->Cell(0,6,"Capítulo $num : $label",0,1,'L',true);
163 // Salto de línea
164 $this->Ln(4);
165 }
166
167 function ChapterBody($file)
168 {
169 // Leemos el fichero
170 $txt = file_get_contents($file);
171 // Times 12
172 $this->SetFont('Times','',12);
173 // Imprimimos el texto justificado

```

- Header() ○ Personaliza el encabezado del PDF.
 - Logo: Inserta una imagen en la esquina superior izquierda (../images/UPSLPlogo.png) con dimensiones ajustadas (33 píxeles).
 - Fuente: Configura la fuente del texto a Arial, estilo negrita (B), tamaño 15.
 - Título: Centra el texto "Reporte de avance" en el encabezado.
 - Espaciado: Agrega un salto de línea de 20 unidades para dejar espacio entre el encabezado y el contenido
- ChapterTitle(\$num, \$label)
 - Define un título para un capítulo del PDF.
 - Fuente: Configura la fuente a Arial, estilo normal, tamaño 12.
 - Color de Fondo: Establece un fondo de color azul claro (SetFillColor(200, 220, 255)) para el título.
 - Texto: Escribe "Capítulo X: [título]" alineado a la izquierda.
 - Espaciado: Agrega un salto de línea de 4 unidades para separar el título del contenido.
- ChapterBody(\$file) ○ Imprime el cuerpo del capítulo a partir de un archivo de texto.
 - Leer Archivo: Utiliza file_get_contents para cargar el contenido del archivo especificado.

- Fuente: Configura la fuente a Times, estilo normal, tamaño 12.
- Texto Justificado: Usa MultiCell para imprimir el texto en varias líneas con alineación justificada.
- Cita Final: Añade un mensaje "(fin del extracto)" en estilo itálico al final del texto.

```

estadísticaPDF.php X
archivosPHP > estadísticaPDF.php
168 {
173 // Imprimimos el texto justificado
174 $this->MultiCell(0,5,$txt);
175 // Salto de línea
176 $this->Ln();
177 // Cita en itálica
178 $this->SetFont('', 'I');
179 $this->Cell(0,5,'(fin del extracto)');
180 }
181
182 function PrintChapter($num, $title, $file)
183 {
184     $this->AddPage();
185     $this->ChapterTitle($num,$title);
186     $this->ChapterBody($file);
187 }
188 // Cargar los datos
189 function LoadData($file)
190 {
191     // Leer las líneas del fichero
192     $lines = file($file);
193     $data = array();
194     foreach($lines as $line)
195     {
196         $data[] = explode(';',trim($line));
197     }
198     return $data;
199 }
200 // Tabla simple
201 function BasicTable($header, $data)
202 {
203     // Cabecera
204     $i=1;
205     foreach($header as $col){
206         if($i==1)
207             $this->Cell(120,6,utf8_decode($col),1,0,'C');
208         else
209             $this->Cell(50,6,utf8_decode($col),1,0,'C');
210     }
211 }

```

- PrintChapter(\$num, \$title, \$file)
 - Combina los métodos anteriores para crear un capítulo completo en el PDF.
 - Añadir Página: Crea una nueva página para el capítulo.
 - Título del Capítulo: Llama a ChapterTitle para añadir un título.
 - Cuerpo del Capítulo: Llama a ChapterBody para imprimir el contenido del archivo.
- LoadData(\$file)
 - Carga datos desde un archivo y los organiza en un arreglo.
 - Leer Archivo: Usa file para leer todas las líneas del archivo.
 - Procesar Líneas: Divide cada línea en columnas separadas por el carácter ; y elimina espacios en blanco al principio y final.
 - Resultado: Devuelve un arreglo bidimensional donde cada línea es una fila y cada columna es un valor.
- BasicTable(\$header, \$data)
 - Genera una tabla simple con encabezados y datos.
 - Encabezado: Imprime los títulos de las columnas (\$header) con alineación centrada (C) y con diferentes anchos para la primera y las demás columnas.
 - Datos: Recorre el arreglo \$data (si es válido) y genera filas en la tabla.

- Usa utf8_decode para manejar caracteres especiales como tildes y ñ. •

```

209     $i++;
210 }
211 $this->Ln();
212 // Datos
213 if (is_array($data) || is_object($data)) {
214     foreach ($data as $row) {
215         {
216             $i=1;
217             foreach($row as $col){
218                 if($i==1)
219                     $this->Cell(120,6,utf8_decode($col),1);
220                 else
221                     $this->Cell(60,6,utf8_decode($col),1);
222                 $i++;
223             }
224             $this->Ln();
225         }
226     }
227 } else {
228     echo "No hay datos para procesar.";
229 }
230 }
231
232 function ProgressBar($datos)
233 {
234     $largo = $datos['largo']; // Largo de la barra calculado previamente
235     $porcentaje = $datos['porcentaje']; // Porcentaje calculado previamente
236     $restante = $datos['restante'];
237
238     $this->Ln(8); // Espacio entre barras
239
240     // Mostrar barra de progreso
241     $this->SetFillColor(21, 40, 116); // Azul para la barra
242     $this->Cell($largo, 6, $porcentaje . '%', 0, 0, 'C', true);

```

- Método ProgressBar ○ Barra de progreso: Crea una barra visual que representa el progreso del alumno en porcentaje.
 - largo: Representa la longitud proporcional al porcentaje.
 - restante: Espacio que queda para completar la barra (180 - \$largo).
 - Estilo:
 - Usa colores:
 - Azul para la parte completada.
 - Blanco para la parte restante.
 - Agrega el porcentaje dentro de la barra.
 - Espaciado: Separa visualmente la barra del resto del contenido con Ln(8) antes y después.

```

242 $this->Cell($largo, 6, $porcentaje . '%', 0, 0, 'C', true);
243 $this->SetFillColor(255, 255, 255);
244 $this->Cell($restante, 6, '', 1, 0, 'C', true);
245
246 $this->Ln(8); // Espacio entre barras
247 }
248
249
250 }
251
252 $pdf = new PDF();
253 $pdf->AliasNbPages();
254 $pdf->AddPage();
255 $pdf->SetFont('Times', '', 12);
256 $title = 'REPORTE DE AVANCE';
257 $pdf->SetTitle($title);
258 $header = ['Materia', 'Creditos'];
259
260 $subtitle = ['PROGRESO DEL ALUMNO'];
261 $total = count($alumnosConCreditos); // Total de elementos
262 $posicion = 0; // Índice inicial
263
264 foreach ($alumnosConCreditos as $nombre => $datos) {
265
266     $posicion++;
267
268     // Cambiar el color de fondo
269     $pdf->SetFillColor(231, 120, 23);
270     // Dibujar la celda con color de fondo y texto
271     $pdf->Cell(180, 10, "Alumno: " . utf8_decode($nombre), 1, 1, 'C', true); // true para aplicar el fondo
272
273     // Cambiar el color del texto
274     $pdf->SetTextColor(255, 255, 255); // Texto blanco
275
276

```

- Inicialización del PDF
 - AliasNbPages: Activa la función para mostrar el total de páginas en el pie de página.
 - AddPage: Crea la primera página del documento.
 - SetFont: Define la fuente inicial como Times de tamaño 12.
 - Header de tablas: Define el encabezado para las tablas que listarán las materias aprobadas y faltantes.
- Generación de Datos por Alumno
 - Color de Fondo: Establece un color de fondo naranja para la celda que contiene el nombre del alumno.
 - Nombre del Alumno: Crea una celda centrada con el nombre del alumno.
 - \$pdf->ProgressBar(\$datos). Muestra la barra de progreso calculada previamente con los datos de cada alumno.

```

277 // Mostrar barra de progreso
278 $pdf->ProgressBar($datos);
279
280
281 // Cambiar el color del texto verde
282 $pdf->SetTextColor(0, 150, 0);
283
284 // Tabla de créditos obtenidos
285 $pdf->Cell(192, 10, 'CREDITOS OBTENIDOS', 0, 10, 'C');
286
287 $pdf->SetTextColor(0, 0, 0); // Texto negro
288
289 // Iterate through approved subjects (materiasAprobadas)
290 $creditosOb = [];
291 foreach ($datos['materiasAprobadas'] as $materia) {
292     $creditosOb[] = ['materia' => $materia['materia'], 'creditos' => $materia['creditos']];
293 }
294 $pdf->BasicTable($header, $creditosOb);
295
296 $pdf->SetTextColor(180, 0, 0); //Cambiar el color del texto rojo
297
298 // Tabla de créditos faltantes
299 $pdf->Cell(192, 10, 'CREDITOS FALTANTES', 0, 10, 'C');
300 // Comparar con el plan de estudios ($creditos) para determinar materias faltantes
301
302 $pdf->SetTextColor(0, 0, 0); // Texto negro
303 // Iterate through missing subjects (materiasFaltantes)
304
305 $creditosF = [];
306 if (isset($alumnosConCreditos[$nombre]['materiasFaltantes'])) {
307     foreach ($alumnosConCreditos[$nombre]['materiasFaltantes'] as $materiaFaltante) {
308         $creditosF[] = [
309             'materia' => $materiaFaltante['materia'],
310             'creditos' => $materiaFaltante['creditos']
311         ];
312     }
313 }
314
315 $pdf->BasicTable($header, $creditosF);
316
317
318 if($posicion < $total){
319     $pdf->AddPage(); // Newpage after each student
320 }
321
322
323
324
325 }
326
327 $pdf->Output();
328 $conn->close();
329
330 ?>
331
332
333

```

- Tabla de Créditos Obtenidos ○ Título: Muestra el texto "CREDITOS OBTENIDOS" en verde. ○
 - Datos: Recorre las materias aprobadas y las estructura en un arreglo \$creditosOb.
 - Tabla: Llama a BasicTable para generar una tabla con las materias aprobadas y sus respectivos créditos.
- Tabla de Créditos Faltantes ○ Título: Muestra "CREDITOS FALTANTES" en rojo.
 - Datos: Recorre las materias faltantes y las estructura en un arreglo \$creditosF. ○
 - Tabla: Llama a BasicTable para listar las materias faltantes y sus créditos

```

311 ];
312 }
313 }
314
315
316 $pdf->BasicTable($header, $creditosF);
317
318
319 if($posicion < $total){
320     $pdf->AddPage(); // Newpage after each student
321 }
322
323
324
325 }
326
327 $pdf->Output();
328 $conn->close();
329
330 ?>
331
332
333

```

- \$pdf->AddPage(); Añade una nueva página después de cada alumno
- \$pdf->Output(); \$conn->close(); Output: Genera el archivo PDF y lo envía al navegador. Cerrar Conexión: Libera recursos al cerrar la conexión a la base de datos.

7. REPORTE.PHP

- 2 Control de acceso: Protege las páginas restringidas verificando si la sesión está activa.
- 3 Redirecciones: Los usuarios son redirigidos a la página adecuada según su rol o a "acceso denegado" si el rol no es permitido.
- 4 Conexión externa: Usa un archivo conexion.php para mantener centralizada la configuración de la base de datos.
- 5 Codificación UTF-8: Esto asegura que los datos se procesen correctamente, especialmente caracteres especiales.

```
<?php
session_start();

// Verifica si la sesión está iniciada
if (!isset($_SESSION['tipo'])) {
    // Si no está iniciada la sesión, redirige al login
    header('Location: https://orangered-donkey-303339.hostingersite.com/proyectoFinal/login.php'); // Cambia 'login.php' por
    exit(); // Detiene la ejecución del script para evitar que se siga ejecutando el resto del código
} else {
    $role = $_SESSION['tipo']; // Obtén el rol del usuario desde la sesión
}

// Determina la URL de inicio según el rol
if ($role == '0') {
    $inicioUrl = '../paginaA.php'; // Página de inicio para administradores
} else {
    if ($role == '1') {
        $inicioUrl = '../paginaM.html'; // Página de inicio para moderadores
    } else {
        header('Location: acceso_denegado.php'); // Redirige a una página de acceso denegado
        exit();
    }
}

// Conexión a la base de datos
$conn = require("conexion.php");
$conn->set_charset("utf8");
```

Recepción segura: Usa `real_escape_string` para proteger la entrada del usuario contra inyecciones SQL.

Validación de existencia: Consulta si la matrícula está registrada en la tabla kardex.

Redirección: Si la matrícula existe, abre un PDF con los datos del kardex; de lo contrario, muestro un mensaje.

```
// Recupera y valida la matrícula del formulario
$matriculaAl = isset($_POST['matricula']) ? $conn->real_escape_string($_POST['matricula']) : "";

$mensaje = ""; // Inicializa el mensaje

if ($matriculaAl) {
    // Consulta para verificar si la matrícula existe en el kardex
    $sql = "SELECT matricula FROM kardex WHERE matricula = '$matriculaAl'";
    $result = $conn->query($sql);

    if ($result->num_rows > 0) {
        // Redirige si se encuentra la matrícula
        $_SESSION['matriculaAlumno'] = $matriculaAl;
        echo "<script>
        window.open('./kardexPDF.php', '_blank');
        </script>";
    } else {
        // Muestra mensaje si no se encuentra la matrícula
        $mensaje = "Kardex no dado de alta.";
    }
}
?>
```

Barra de Navegación

Incluye enlaces para regresar al inicio, cerrar sesión y acceder a la página de ayuda. La URL de inicio depende del rol del usuario.

```
<body>

    <!-- Barra de navegación -->
    <nav class="barra">
        <ul class="topnav">
            <li><a href="<?php echo $inicioUrl; ?>">Inicio</a></li>
            <li><a href=" ../archivosPHP/logout.php">Logout</a></li>
            <li class="right"><a href=" ../archivosPHP/ayuda.php">Help</a></li>
        </ul>
    </nav>
```

Formulario: Permite al usuario ingresar su matrícula para consultar el kardex.

Validación: Usa required para asegurar que el campo no esté vacío.

Mensajes: Los errores o resultados se muestran dinámicamente.

```

<!-- Formulario en tarjeta central -->
<div class="card">
  <h2>Kardex de Alumno</h2>
  <?php if ($mensaje): ?> <!-- Muestra el mensaje si existe -->
    <p style="color: red; font-weight: bold;"><?php echo $mensaje; ?></p>
  <?php endif; ?>
  <form action="reporte.php" method="post">
    <label for="matricula">Ingresa tu matrícula:</label>
    <input type="text" id="matricula" name="matricula" placeholder="Ej. 123456" required>
    <button type="submit">Ver Kardex</button>
  </form>
</div>

</body>
</html>

```

8. GENERACION DE KARDEX EN PDF

8.1 PHP

```

archivosPHP > kardexPDF.php
1  <?php
2  session_start();
3
4  if( $_SESSION['tipo']==1 || $_SESSION['tipo']== 0){
5      $matricula_alumno= $_SESSION['matriculaAlumno'];
6  }else if( $_SESSION['tipo']==2){
7      $matricula_alumno= $_SESSION['usuario'];
8  }
9
10 require('fpdf183/fpdf.php');
11 $conn=require("conexion.php");
12 $conn->set_charset("utf8");
13
14 $aprobadas = array();
15 $actuales = array();
16 $siguientes = array();
17
18 // Consulta para determinar la carrera
19 $sql = "SELECT carrera FROM kardex WHERE matricula = ?";
20 $stmt = $conn->prepare($sql);
21 $stmt->bind_param("s", $matricula_alumno); // 's' indica que es un string
22 $stmt->execute();
23 $result = $stmt->get_result();
24
25 if ($result->num_rows > 0) {
26     $row = $result->fetch_assoc();
27     $carrera = trim(strtoupper($row['carrera']));
28
29     if ($carrera == 'ITI01') {
30         $planEstudios = "plan_estudios";
31         $seriaciones = "seriaciones";
32     } elseif ($carrera == 'ITEM02') {
33         $planEstudios = "plan_estudios_item";
34         $seriaciones = "seriaciones_item";
35     } else {
36         echo "Carrera no reconocida: " . $carrera;
37     }

```

- session_start(); Inicia la sesión para acceder a las variables de sesión (\$_SESSION), donde están almacenados datos del usuario autenticado, como su matrícula y tipo de usuario.
- Determinación de la Matrícula. Dependiendo del tipo de usuario (tipo), asigna la matrícula a la variable \$matricula_alumno.
 - Tipo 1 o 0: Accede a matriculaAlumno.
 - Tipo 2: Accede a usuario.
- \$conn = require("conexion.php"); \$conn->set_charset("utf8"); Incluye el archivo conexion.php para establecer la conexión con la base de datos.

- Codificación: Configura la conexión para utilizar el juego de caracteres utf8, asegurando que los datos se manejen correctamente en español (acentos, ñ, etc.).
- Consulta para Determinar la Carrera. Obtiene la carrera del alumno desde la tabla kardex utilizando su matrícula.
 - Uso de Prepared Statements: Mejora la seguridad contra inyecciones SQL mediante la preparación de la consulta y el uso de parámetros vinculados.

Procesamiento del Resultado.

- Verifica si hay resultados: ○ Si encuentra datos, extrae la carrera asociada al alumno.
- Limpieza y Transformación:
 - Utiliza trim para eliminar espacios y strtoupper para convertir el texto a mayúsculas, asegurando uniformidad.
- Condicionales por Carrera:
 - Si la carrera es ITIO1, asigna los nombres de las tablas relevantes a las variables:
 - ▢ \$planEstudios → Tabla de plan de estudios.
 - ▢ \$seriaciones → Tabla de seriaciones.
- Si la carrera es ITEM02, asigna las tablas correspondientes.
- En caso de que la carrera no sea reconocida, muestra un mensaje de error.
- Sin Resultados:
 - Si no se encuentra la carrera en la base de datos, muestra un mensaje indicando que no hay datos asociados a la matrícula.

```

42 // Consulta para obtener las materias aprobadas
43 $sql = "SELECT * FROM kardex WHERE status='Aprobado' AND matricula= '$matricula_alumno' ";
44 $result = $conn->query($sql);
45 if ($result->num_rows > 0) {
46     // Recorre cada fila de resultados
47     while ($row = $result->fetch_assoc()) {
48         // Guarda cada fila como un arreglo asociativo con detalles adicionales
49         array_push($aprobadas, $row["materia"]);
50     }
51 }
52
53 // Consulta para obtener las materias actuales guardadas por clave de la materias
54 $sql = "SELECT * FROM kardex WHERE periodo='20243S' AND matricula= '$matricula_alumno' ";
55 $result = $conn->query($sql);
56 if ($result->num_rows > 0) {
57     while ($row = $result->fetch_assoc()) {
58         array_push($actuales, $row["materia"]);
59     }
60 }
61
62 // Fusionar los arreglos $aprobadas y $actuales
63 $materias_cursadas = array_merge($aprobadas, $actuales);
64
65 // Crear la cadena separada por comas
66 $cursadas_str = implode(",", $materias_cursadas);
67
68 $sql = " SELECT * FROM kardex
69 JOIN $seriaciones ON TRIM(LOWER(kardex.materia)) = TRIM(LOWER($seriaciones.nombreMateria))
70 WHERE kardex.matricula = '$matricula_alumno'
71 AND kardex.periodo != '20243S'
72 AND kardex.materia NOT IN ('$cursadas_str')
73 AND (

```

- Consulta de Materias Aprobadas. Obtiene todas las materias aprobadas por el alumno basándose en el campo status='Aprobado'.
 - Resultado: Cada materia aprobada se agrega al arreglo \$aprobadas.
- Consulta de Materias Actuales (en Curso). Filtra las materias que el alumno está cursando actualmente según el periodo='20243S'.
 - Resultado: Cada materia en curso se agrega al arreglo \$actuales.
- Fusión de Materias Aprobadas y Actuales
 - array_merge: Combina las materias aprobadas (\$aprobadas) y las actuales (\$actuales) en un solo arreglo llamado \$materias_cursadas.
 - implode: Convierte el arreglo resultante en una cadena separada por comas y apóstrofes (','), que se utilizará en una consulta SQL posterior.
- Consulta de Materias recomendadas para el próximo semestre
- Condiciones principales:
 - Materias no cursadas: kardex.materia NOT IN ('\$cursadas_str'). Excluye materias que ya están en el arreglo de materias cursadas (aprobadas o actuales).
 - Prerrequisitos cumplidos:
 - La materia tiene como prerrequisito alguna de las materias cursadas (TRIM(LOWER(\$seriaciones.nombrePre)) IN ('\$cursadas_str')).
 - O no tiene prerrequisitos (\$seriaciones.nombrePre = '-').
- Filtro de periodo: Excluye materias en el periodo actual (kardex.periodo != '20243S').
- Estado del Kardex:
 - Materias sin información de periodo (kardex.periodo IS NULL AND kardex.periodo = '').
 - O materias con estado no definido (kardex.status = '-').
- Orden y Límite:
 - Ordena por el semestre al que pertenecen (ORDER BY kardex.sem).
 - Limita la salida a 7 materias (LIMIT 7).

```
73  AND (  
74      TRIM(LOWER($seriaciones.nombrePre)) IN ('$cursadas_str')  
75      OR $seriaciones.nombrePre = '-'  
76  )  
77  AND (  
78      (kardex.periodo IS NULL AND kardex.periodo = '')  
79      OR (kardex.periodo IS NOT NULL AND kardex.status = '-')  
80  )  
81  ORDER BY kardex.sem  
82  LIMIT 7  
83  ";  
84  
85  //arreglo de materias siguientes guardadas por clave de materia  
86  $result = $conn->query($sql);  
87  
88  if ($result->num_rows > 0) {  
89      // Recorrer los resultados y agregarlos al arreglo  
90      while ($row = $result->fetch_assoc()) {  
91          $siguientes[] = $row['materia'];  
92      }  
93  }
```

- Almacenamiento de Materias Siguietes. Ejecuta la consulta para obtener las materias siguietes y las almacena en el arreglo \$siguietes.

```

162 $pdf = new PDF();
163 $pdf->AliasNbPages();
164 $pdf->AddPage('O');
165 $pdf->SetFont('Times','',12);
166 $title = 'Reporte '.$matricula_alumno;
167 $pdf->SetTitle($title);
168 //utf8_decode($col)
169
170 $cont=0;
171 $still=0;
172 $sem = 0;
173 $pdf->SetFont('Times','',8);
174 $conn->set_charset("utf8");
175
176 $pdf->SetFontSize(10);
177 $pdf->SetXY(0,20);
178 $pdf->Cell(120,10,'Matricula: '.$matricula_alumno,0,0,'C',0);
179 $pdf->SetFontSize(10);
180
181 $pdf->SetFillColor(0,255,0);
182 $pdf->Cell(5,5,'',0,0,'C',TRUE);
183 $pdf->SetFillColor(255,255,255);
184 $pdf->Cell(25,5,'Aprobadas',0,0,'C',TRUE);
185
186 $pdf->SetFillColor(255, 182, 193);
187 $pdf->Cell(5,5,'',0,0,'C',TRUE);
188 $pdf->SetFillColor(255,255,255);
189 $pdf->Cell(35,5,'Cursadas actualmente',0,0,'C',TRUE);
190
191 $pdf->SetFillColor(0, 255, 255);
192 $pdf->Cell(5,5,'',0,0,'C',TRUE);
193 $pdf->SetFillColor(255,255,255);
194 $pdf->Cell(20,5,'Sugeridas',0,0,'C',TRUE);

```

- Configuración Inicial PDF
 - Clase PDF: Se instancia un objeto de la clase PDF (previamente definida).
 - AliasNbPages: Configura un marcador que permite usar {nb} en la numeración de páginas.
 - AddPage('O'): Agrega una nueva página al PDF en orientación horizontal (O para landscape).
 - SetFont: Define la fuente como Times con estilo normal y tamaño 12.
 - SetTitle: Establece el título del documento, incluyendo la matrícula del alumno.
- Información del Alumno
 - SetFontSize: Cambia el tamaño de la fuente a 10.
 - SetXY(0,20): Posiciona el cursor de impresión en las coordenadas (0, 20) en la página.
 - Cell: Genera una celda que contiene el texto Matricula: {matricula_alumno} centrado horizontalmente ('C').
- Leyenda de Colores
 - Se crean celdas pequeñas de 5x5 píxeles rellenas con colores que representan diferentes estados de las materias:
 - Verde (SetFillColor(0,255,0)): Materias aprobadas.
 - Rosa (SetFillColor(255, 182, 193)): Materias cursadas actualmente.
 - Cian (SetFillColor(0, 255, 255)): Materias sugeridas para cursar próximamente.
 - Amarillo (SetFillColor(255,255,1)): Materias sin cursar.


```

194 $pdf->Cell(20,5,'Sugeridas',0,0,'C',TRUE);
195
196 $pdf->SetFillColor(255,255,1);
197 $pdf->Cell(5,5,'',0,0,'C',TRUE);
198 $pdf->SetFillColor(255,255,255);
199 $pdf->Cell(20,5,'Sin cursar',0,1,'C',TRUE);
200
201 $pdf->Ln(5);
202
203 $materias_filtradas= array();
204 $materias_kardex = array();
205 $materias_plan= array();
206

```

```

203 $materias_filtradas= array();
204 $materias_kardex = array();
205 $materias_plan= array();
206
207
208 // Realiza la consulta para obtener datos iniciales
209 $sql = "SELECT * FROM kardex WHERE sem < 10 AND matricula='\$matricula_alumno' ORDER BY id ASC";
210 $result = $conn->query($sql);
211
212 // Arreglo para guardar los registros
213 $registros = array();
214
215 // Construye el arreglo inicial
216 if ($result->num_rows > 0) {
217     while ($row = $result->fetch_assoc()) {
218         $registros[] = $row;
219     }
220 }
221
222 $materias = [];
223 foreach ($registros as $registro) {
224     $materia = $registro['materia'];
225     $status = $registro['status'];
226     $periodo = $registro['periodo'];
227
228     // Si no existe o no es "Aprobado", actualiza
229     if (!isset($materias[$materia]) || $materias[$materia]['status'] !== 'Aprobado') {
230         $materias[$materia] = [
231             'status' => ($status === 'Aprobado') ? 'Aprobado' : '-',
232             'periodo' => $periodo
233         ];
234     }
235 }
236

```

- Consulta de Datos Iniciales ○ Selecciona todas las materias del estudiante cuya matrícula coincide con \$matricula_alumno.
 - Filtra las materias de semestres menores a 10 (sem < 10).
 - Ordena los resultados por el campo id en orden ascendente.
- Almacenamiento en \$registros:
 - Cada fila obtenida de la base de datos se guarda como un arreglo asociativo en \$registros.
- Procesamiento de Materias ○ Iteración sobre \$registros:
 - Para cada materia (\$materia), verifica si ya existe en \$materias.
 - Si no existe o si el estado de la materia no es "Aprobado", actualiza o agrega el registro.
 - Estructura del Arreglo \$materias:
 - Cada clave del arreglo corresponde al nombre de una materia.
 - Cada materia tiene dos atributos:
 - status: Indica si la materia está "Aprobada" o no ('-').

- periodo: Almacena el periodo correspondiente a la materia.

```

237
238 // Arreglo para agrupar materias por semestre
239 $semestres = [];
240 foreach ($registros as $registro) {
241     $materia = $registro['materia'];
242     $sem = $registro['sem'];
243
244     // Aseguramos que el semestre exista en el arreglo
245     if (!isset($semestres[$sem])) {
246         $semestres[$sem] = [];
247     }
248
249     // Usamos el estado procesado de la materia
250     if (isset($materias[$materia])) {
251         $semestres[$sem][$materia] = $materias[$materia];
252     }
253 }
254
255 // Ordenamos los semestres
256 ksort($semestres);
257

```

- Agrupación por Semestre Iteración sobre \$registros:
 - Para cada registro, se obtiene el semestre (\$sem) y la materia.
 - Si el semestre aún no está definido en \$semestres, lo inicializa como un arreglo vacío.
- Asignación de Materias:
 - Si la materia existe en \$materias, se agrega al semestre correspondiente en \$semestres.
- ksort: Ordena los semestres por su clave de forma ascendente, asegurando que los semestres estén en el orden correcto (1, 2, 3, ...).

```

258 // Genera la tabla con los datos procesados
259 $sem_actual = 0;
260 foreach ($semestres as $sem => $materias_semestre) {
261     if ($sem_actual != $sem) {
262         if ($sem_actual != 0) {
263             $pdf->Cell(1, 15, '', 0, 1, 'C', 0);
264         }
265
266         $pdf->SetFillColor(215, 234, 255);
267         $pdf->Cell(20, 15, utf8_decode($sem . 'º Sem'), 1, 0, 'C', true);
268         $pdf->SetFillColor(255, 255, 255);
269         $sem_actual = $sem;
270     }
271
272     // Usa $materias_semestre en lugar de $materias
273     foreach ($materias_semestre as $materia => $datos) {
274         $status = $datos['status'];
275         $periodo = $datos['periodo'];
276
277         $pdf->SetFillColor(255, 255, 1);
278
279         if ($status === "Aprobado") {
280             $pdf->setTextColor(11, 63, 71);
281             $pdf->SetFillColor(0, 255, 0);
282         }
283
284         if ($periodo === "20243S") {
285             $pdf->SetFillColor(255, 182, 193); // Fondo rosa claro
286             $pdf->setTextColor(0, 0, 0);
287         }
288         if (in_array($materia, $siguientes)) {
289             $pdf->SetFillColor(0, 255, 255);
290             $cont++;
291         }
292     }
293 }

```

Generar tabla con los datos procesados

- Iteración sobre los Semestres
 - Se recorre el arreglo \$semestres, donde la clave \$sem es el semestre y
 - \$materias_semestre contiene las materias correspondientes a ese semestre.
 - Si el semestre actual (\$sem_actual) es diferente al semestre actual en el ciclo (\$sem), se agrega una celda con el nombre del semestre (\$sem).
 - Se cambian los colores de fondo de la celda para indicar visualmente un nuevo semestre.
- Iteración sobre las Materias del Semestre. Segundo ciclo foreach sobre las materias dentro de un semestre:
 - Se recorre cada materia dentro del semestre actual, utilizando \$materia como clave y \$datos para obtener el estado (status) y periodo (periodo) de la materia.
- Establecimiento de Colores Según el Estado de la Materia. Se asignan diferentes colores de fondo dependiendo del estado de la materia:
 - Verde para las materias aprobadas.
 - Rosa claro para las materias cursadas en el periodo actual (20243S).
 - Cian para las materias que están en el arreglo \$siguientes, que son las sugeridas o faltantes.
- Configuración de la Celda para la Materia
 - Ajuste de tamaño de celda:
 - ▢ \$Inwidth obtiene el ancho de la cadena de texto (nombre de la materia).
 - ▢ Si el ancho de la cadena es mayor que 37.5, la altura de la celda se ajusta a 7.5, de lo contrario, se establece en 15.
 - MultiCell: Se utiliza para escribir la materia en una celda que puede contener texto de varias líneas, con un tamaño de celda determinado por \$heightCell.
 - ▢ La celda tiene un borde, y el texto está centrado.
 - Ajuste de la posición para la siguiente celda:
 - ▢ SetXY reposiciona el cursor en la siguiente celda.

```

292 // Configuración de la celda
293 $lwidth = $pdf->GetStringWidth($materia);
294 $heightCell = ($lwidth > 37.5) ? 7.5 : 15;
295
296 $current_y = $pdf->GetY();
297 $current_x = $pdf->GetX();
298
299 $pdf->MultiCell(37.5, $heightCell, utf8_decode($materia), 1, 'C', true);
300 $pdf->SetXY($current_x + 37.5, $current_y);
301
302 }
303
304 }
305
306 $pdf->SetXY(10, $current_y+17);
307
308 $sql = "SELECT SUM(creditos) AS total FROM kardex WHERE matricula= '$matricula_alumno' ";
309 $result = $conn->query($sql);
310
311 if ($result->num_rows > 0) {
312     $row = $result->fetch_assoc();
313     // Verifica si el total es NULL y asigna 0 si es el caso
314     $creditost = $row['total'] ? $row['total'] : 0;
315 } else {
316     // Si no hay resultados, asigna 0
317     $creditost = 0;
318 }
319
320 $porcentaje = round( ($creditost*100)/408 );
321 $largo = ($porcentaje*280)/100;
322 $restante = 280 - $largo;
323
324 $pdf->SetFontSize(10);
325 $pdf->Cell(280,10,'PROGRESO DE CREDITOS DEL ALUMNO',0,1,'C',0);
326 $pdf->SetFillColor(231,120,23);
327 $pdf->Cell($largo,8,$porcentaje.'%',1,0,'C',TRUE);
328 $pdf->Cell($restante,8,'',1,1,'C',FALSE);
329
330
331
332 $pdf->Output();
333

```

- Calcular el Total de Créditos Obtenidos
 - Se realiza una consulta SQL para obtener el total de créditos obtenidos por el alumno en la base de datos.
 - Si se encuentra un resultado, se verifica que no sea NULL y se asigna el total de créditos a la variable \$creditost. Si el resultado es NULL, se asigna 0.
 - Si no se encuentran resultados, se asigna 0 a \$creditost.
- Calcular el Porcentaje de Créditos Completados.
 - Se calcula el porcentaje de créditos completados dividiendo los créditos obtenidos (\$creditost) por el total de créditos requeridos (en este caso, 408), y multiplicando el resultado por 100. El resultado se redondea a un valor entero.
- Calcular el Tamaño de la Barra de Progreso.
 - El largo de la barra de progreso se calcula en función del porcentaje de créditos completados. Se ajusta el valor del largo a un máximo de 280 (que es el ancho de la barra de progreso en píxeles).
 - La variable \$restante calcula el espacio restante para completar la barra (es decir, el total menos el largo de la barra).
- Configuración y Generación de la Barra de Progreso en el PDF
 - Título: Se muestra un título centrado: 'PROGRESO DE CREDITOS DEL ALUMNO'.
 - Barra de Progreso: Se establece el color de fondo de la barra utilizando SetFillColor(231,120,23) (un tono de naranja).
 - La barra se crea con una celda que tiene un ancho proporcional al porcentaje de créditos completados (\$largo).
 - La celda que queda vacía (\$restante) se completa con un espacio vacío.

9. VERKARDEX.PHP

1. Inicio de sesión: Usa `session_start()` para habilitar el manejo de sesiones.
2. Protección de acceso: Verifica el tipo de usuario mediante `$_SESSION['tipo']`.
3. Redirecciones seguras: Los usuarios son redirigidos según su rol o a una página de acceso denegado si no tienen permisos.

```
<?php
session_start();

// Verifica si la sesión está iniciada
if (!isset($_SESSION['tipo'])) {
    header('Location: https://orangered-donkey-303339.hostingersite.com/proyectoFinal/login.php');
    exit();
} else {
    $role = $_SESSION['tipo'];
}

// Determina la URL de inicio según el rol
if ($role == '0') {
    $inicioUrl = '../paginaA.php';
} else {
    if ($role == '1') {
        $inicioUrl = '../paginaM.html';
    } else {
        header('Location: https://orangered-donkey-303339.hostingersite.com/proyectoFinal/archivosPHP/acceso_denegado.php');
        exit();
    }
}
}
```

1. Uso de un archivo externo: Mantiene centralizada la configuración de la conexión.
2. Configuración de codificación: Asegura el manejo correcto de caracteres especiales mediante UTF-8.

```
// Conexión a la base de datos
$conn = require("conexion.php");
$conn->set_charset("utf8");
```

1. Recepción de datos: Obtenga la matrícula del formulario mediante `$_POST`.
2. Consulta a la base de datos: Verifica si la matrícula existe en la tabla kardex.
3. Redirección: Si la matrícula es válida, redirige al archivo `verKardexAlumno.php`. Si no, muestra un mensaje de error.

```
// Recupera y valida la matrícula del formulario
$matriculaAl = $_POST['matricula'];

$mensaje = ""; // Inicializa el mensaje

if ($matriculaAl) {
    // Consulta para verificar si la matrícula existe en el kardex
    $sql = "SELECT matricula FROM kardex WHERE matricula = '$matriculaAl'";
    $result = $conn->query($sql);

    if ($result->num_rows > 0) {
        // Redirige si se encuentra la matrícula
        $_SESSION['matriculaAlumno'] = $matriculaAl;
        header('Location: ../verKardexAlumno.php');
        exit();
    } else {
        // Muestra mensaje si no se encuentra la matrícula
        $mensaje = "Kardex no dado de alta.";
    }
}
?>
```

Barra de Navegación

Proporciona enlaces para navegar al inicio, cerrar sesión y acceder a la ayuda. La URL de inicio se ajusta según el rol del usuario.

```
<!-- Barra de navegacion -->
<nav class="barra">
    <ul class="topnav">
        <li><a href="<?php echo $inicioUrl; ?>">Inicio</a></li>
        <li><a href=" ../archivosPHP/logout.php">Logout</a></li>
        <li class="right"><a href=" ../archivosPHP/ayuda.php">Help</a></li>
    </ul>
</nav>
```

1. Formulario: Permite al usuario ingresar su matrícula.
2. Mensajes: Los errores o resultados se muestran dinámicamente.
3. Validación en cliente: Uso required para garantizar que el campo no esté vacío.

```
<!-- Formulario en tarjeta central -->
<div class="card">
    <h2>Kardex de Alumno</h2>
    <?php if ($mensaje): ?> <!-- Muestra el mensaje si existe -->
        <p style="color: red; font-weight: bold;"><?php echo $mensaje; ?></p>
    <?php endif; ?>
    <form action="verKardex.php" method="post">
        <label for="matricula">Ingresa tu matrícula:</label>
        <input type="text" id="matricula" name="matricula" placeholder="Ej. 123456" required>
        <button type="submit">Ver Kardex</button>
    </form>
</div>
```


10. KARDEX DEL ALUMNO

10.1 PHP

```

1  <?php
2      session_start();
3
4      // Verifica si la sesión está iniciada
5      if (!isset($_SESSION['tipo'])) {
6          // Si no está iniciada la sesión, redirige al login
7          header('location: https://orangered-donkey-303339.hostingersite.com/proyectofinal/login.php');
8          exit(); // Detiene la ejecución del script para evitar que se siga ejecutando el resto del código
9      }
10
11      if( $_SESSION['tipo']==1 || $_SESSION['tipo']== 0){
12          $matricula_alumno= $_SESSION['matriculaAlumno'];
13      }else if( $_SESSION['tipo']==2){
14          $matricula_alumno= $_SESSION['usuario'];
15      }
16
17  ?>
18
19  <!DOCTYPE html>
20  <html lang="es">
21  <head>
22      <meta charset="UTF-8">
23      <meta name="viewport" content="width=device-width, initial-scale=1.0">
24      <title>Kardex Alumno</title>
25      <link rel="stylesheet" href="../styles/kardexStyle.css">
26      <link rel="stylesheet" href="../styles/navM.css">
27  </head>
28
29  <body>
30      <nav class="barra">
31          <ul class="topnav">
32              <?php
33                  // Verifica el tipo de usuario y muestra los elementos adecuados en la barra de navegación
34                  if ($_SESSION['tipo'] == 1) {
35                      // Maestro
36                      echo '<li><a href="../paginaM.html">Inicio</a></li>';
37                  } else if ($_SESSION['tipo'] == 2) {
38                      // Estudiante
39                      echo '<li><a href="../paginaAl.php">Inicio</a></li>';
40                  } else if($_SESSION['tipo'] == 0){
41                      echo '<li><a href="../paginaA.php">Inicio</a></li>';
42                  }
43              ?>
44          </ul>
45      </nav>

```

```

61  <?php
62
63  function normalizarTexto($texto) {
64      // Convertir a mayúsculas
65      $texto = strtoupper($texto);
66      // Eliminar acentos
67      $texto = iconv('UTF-8', 'ASCII//TRANSLIT', $texto);
68      // Eliminar espacios al inicio y al final
69      $texto = trim($texto);
70      return $texto;
71  }
72
73
74  $conn=require("conexion.php");
75  $conn->set_charset("utf8");
76
77  // Consulta para conocer carrera
78
79  $matricula_alumno = trim($matricula_alumno); // Eliminar espacios adicionales
80  $sql = "SELECT carrera FROM kardex WHERE matricula = '$matricula_alumno'";
81  $result = $conn->query($sql);
82
83  if (!$result) {
84      die("Error en la consulta: " . $conn->error);
85  }
86
87  if ($result->num_rows > 0) {
88      $row = $result->fetch_assoc();
89      $carrera = trim(strtoupper($row['carrera']));
90
91      if ($carrera == 'ITI01') {
92          $planEstudios = "plan_estudios";
93          $seriaciones = "seriaciones";
94      } elseif ($carrera == 'ITEM02') {
95          $planEstudios = "plan_estudios_item";
96          $seriaciones = "seriaciones_item";
97      } else {
98          echo "Carrera no reconocida: " . $carrera;
99      }
100  } else {
101      echo "No se encontraron resultados para la matrícula proporcionada.";
102  }
103
104

```

- Normalización de Texto

Convertir el texto a mayúsculas: Asegura que cualquier entrada de texto esté en formato mayúscula.

- Eliminar acentos: Se realiza usando iconv con el parámetro 'ASCII//TRANSLIT', que reemplaza los caracteres con acento por sus equivalentes sin acento.
- Eliminar espacios al inicio y al final: Se usa trim() para limpiar los espacios adicionales.
- Consulta para Obtener la Carrera del Alumno ○ Eliminación de Espacios Adicionales: Se asegura de que no haya espacios adicionales en la variable \$matricula_alumno antes de hacer la consulta.
 - Consulta SQL: Se hace una consulta a la base de datos para obtener el campo carrera de la tabla kardex, donde el matricula coincide con la matrícula del alumno.
- Procesamiento del Resultado de la Consulta ○ Si la carrera es ITI01, se asignan los valores correspondientes a las variables \$planEstudios y \$seriaciones.
 - Si la carrera es ITEM02, se asignan otros valores a esas mismas variables.
 - Si la carrera no es reconocida, se muestra un mensaje indicando que la carrera no es válida.

```
103
104
105 echo "<div class='datos-alumno'>
106     <p> Matrícula: $matricula_alumno </p>
107
108     </div>
109
110     <div class='leyendascont'>
111     <div class='leyendas'>
112         <div class='cl' id='aprobadas'></div>
113         <div class='leyenda'>Aprobadas</div>
114     </div>
115     <div class='leyendas'>
116         <div class='cl' id='sin-cursar'></div>
117         <div class='leyenda'>Sin cursar</div>
118     </div>
119     <div class='leyendas'>
120         <div class='cl' id='actualmente'></div>
121         <div class='leyenda'>Cursadas actualmente</div>
122     </div>
123     <div class='leyendas'>
124         <div class='cl' id='sugeridas'></div>
125         <div class='leyenda'>Sugeridas</div>
126     </div>
127 </div>";
128
129 // Crear los arreglos de materias
130 $materias = array();
131 $aprobadas = array();
132 $actuales = array();
133 $siguientes = array();
134
```

- Información del Alumno (HTML) ○ Información del Alumno: Muestra la matrícula del alumno en un bloque de div con la clase datos-alumno.
 - Leyendas: Crea un conjunto de leyendas para diferenciar las materias según su estatus. Cada leyenda tiene un color diferente, que está asociado con el CSS para darle un color visual distintivo a cada categoría.

```

35 // Consulta para obtener la información de todas las materias
36 $sql = "SELECT * FROM $planEstudios";
37 $result = $conn->query($sql);
38
39 if ($result->num_rows > 0) {
40
41     // Define los reemplazos para la carrera ITI01
42     $reemplazosITI01 = [
43         'Curso del Núcleo Optativo I' => 'CNO I Programación Web I',
44         'Curso del Núcleo Optativo II' => 'CNO II Programación Web II',
45         'CNG III Filosofía y Valores' => 'Curso del Núcleo General III: Filosofía y Valores'
46     ];
47
48     $reemplazosITEM02 = [
49         'CNG I Desarrollo del Pensamiento Crítico' => 'CNGI Desarrollo del Pensamiento Crítico',
50         'Curso del Núcleo Optativo I' => 'CNO I Programación Web',
51         'Curso del Núcleo Optativo II' => 'CNO II Programación Web II',
52         'Curso del Núcleo Optativo III' => 'CNO III Electromagnetismo',
53         'Curso del Núcleo Optativo IV' => 'CNO IV Microondas y Satélites',
54         'Curso del Núcleo Optativo V' => 'CNO V Seguridad Informática'
55     ];
56
57 // Recorre cada fila de resultados
58 while ($row = $result->fetch_assoc()) {
59     $nombreMateria = $row["nombre"];
60
61     if ($carrera == 'ITI01') {
62         // Aplica el reemplazo si hay una coincidencia exacta
63         foreach ($reemplazosITI01 as $buscado => $reemplazo) {
64             if ($nombreMateria == $buscado) { // Verifica coincidencia exacta
65                 $nombreMateria = $reemplazo; // Realiza el reemplazo
66                 break; // Detenemos más reemplazos
67             }
68         }
69     }
70 }

```

- Consulta SQL ○ Consulta para obtener todas las materias desde la tabla que se define dinámicamente con la variable \$planEstudios. Dependiendo de la carrera del alumno, esta variable cambiará para seleccionar la tabla correcta (por ejemplo, plan_estudios o plan_estudios_item).
- Reemplazos según la carrera ○ Cuando la carrera es ITI01, si el nombre de la materia coincide con alguno de los definidos en \$reemplazosITI01, el nombre de la materia será reemplazado por el valor correspondiente.
- Recorre los resultados y aplica los reemplazos.
 - Dentro del bucle while que recorre cada fila del resultado de la consulta, se comprueba si el nombre de la materia coincide con alguna de las claves en los arreglos de reemplazo, y si hay coincidencia, se actualiza el nombre de la materia.

```

169 }
170
171 if ($carrera == 'ITEM02') {
172     // Aplica el reemplazo si hay una coincidencia exacta
173     foreach ($reemplazosITEM02 as $buscado => $reemplazo) {
174         if ($nombreMateria == $buscado) { // Verifica coincidencia exacta
175             $nombreMateria = $reemplazo; // Realiza el reemplazo
176             break; // Detenemos más reemplazos
177         }
178     }
179 }
180
181 // Guarda cada fila como un arreglo asociativo con detalles adicionales
182 $materias[] = array(
183     "clave_materia" => $row["clave"],
184     "semestre_materia" => $row["semestre"],
185     "nombre_materia" => $nombreMateria,
186     "creditos_materia" => $row["creditos"],
187     "horas_materia" => $row["horas"],
188     "plan_materia" => $row["plan"]
189 );
190 }
191 }
192
193

```

Almacenar la información de las materias

Después de realizar los reemplazos, los detalles de cada materia se almacenan en el arreglo \$materias. Este arreglo guarda un arreglo asociativo con información detallada sobre la materia, como su clave, semestre, nombre, créditos, horas y plan.

```
// Consulta para obtener las materias aprobadas
$sql = "SELECT REPLACE(materia, ':', '') AS materia_aprobada
FROM kardex
WHERE status = 'Aprobado' AND matricula = '$matricula_alumno'";
$result = $conn->query($sql);
if ($result->num_rows > 0) {
    // Recorre cada fila de resultados
    while ($row = $result->fetch_assoc()) {
        // Guarda cada fila como un arreglo asociativo con detalles adicionales
        array_push($aprobadas, $row["materia_aprobada"]);
    }
}

// Consulta para obtener las materias actuales guardadas por clave de la materias
$sql = "SELECT * FROM kardex WHERE periodo='20243S' AND matricula = '$matricula_alumno' ";
$result = $conn->query($sql);
if ($result->num_rows > 0) {
    while ($row = $result->fetch_assoc()) {
        array_push($actuales, $row["materia"]);
    }
}

// Fusionar los arreglos $aprobadas y $actuales
$materias_cursadas = array_merge($aprobadas, $actuales);

// Crear la cadena separada por comas
$cursadas_str = implode(",", $materias_cursadas);

$sql = "SELECT REPLACE(materia, ':', '') AS materia_limpia, COUNT(*) AS veces_cursada
FROM kardex
WHERE periodo != ''
AND matricula = '$matricula_alumno'
GROUP BY materia";

$result = $conn->query($sql);

$materiasVeces = []; // Inicializa el arreglo asociativo

if ($result->num_rows > 0) {
    while ($row = $result->fetch_assoc()) {
        // Agrega al arreglo asociativo el nombre de la materia como clave y las veces cursadas como valor
        $materiasVeces[$row["materia_limpia"]] = $row["veces_cursada"];
    }
}
```

- Consulta para obtener las materias aprobadas ○ Esta consulta obtiene las materias que han sido aprobadas por el alumno, eliminando los ":" del nombre de la materia.
 - Los resultados se guardan en el arreglo \$aprobadas usando array_push.
 - Consulta para obtener las materias actuales (materias cursadas en el periodo 20243S) ○ Aquí se obtiene la lista de materias que el alumno está cursando actualmente en el periodo 20243S.
 - Se guardan en el arreglo \$actuales
 - Fusionar las materias aprobadas y las materias actuales.
 - Se fusionan los dos arreglos (\$aprobadas y \$actuales) en un solo arreglo llamado \$materias_cursadas.
- Crear una cadena con las materias cursadas separadas por comas.
- Se convierte el arreglo \$materias_cursadas en una cadena con los nombres de las materias separados por comas, para poder usarlos en una consulta SQL.
- Consulta para obtener el número de veces que cada materia ha sido cursada ○ Esta consulta obtiene el número de veces que cada materia ha sido cursada, agrupando los resultados por materia.

- Almacenar los resultados en un arreglo asociativo ○ El arreglo \$materiasVeces almacena los resultados de la consulta, donde la clave es el nombre de la materia (sin los :) y el valor es el número de veces que la materia ha sido cursada.

```

237
238
239 $sql = " SELECT * FROM kardex
240 JOIN $seriaciones ON kardex.materia = $seriaciones.nombreMateria
241 WHERE kardex.matricula = '$matricula_alumno'
242 AND kardex.periodo != '202435'
243 AND kardex.materia NOT IN ('$cursadas_str')
244 AND (
245     $seriaciones.nombrePre IN ('$cursadas_str')
246     OR $seriaciones.nombrePre = '-'
247 )
248 AND (
249     (kardex.periodo IS NULL AND kardex.periodo = '')
250     OR (kardex.periodo IS NOT NULL AND kardex.status = '-')
251 )
252 ORDER BY kardex.sem
253 LIMIT 7
254 ";
255
256 //arreglo de materias siguientes guardadas por clave de materia
257 $result = $conn->query($sql);
258 if ($result->num_rows > 0) {
259     while($row = $result->fetch_assoc()){
260         array_push($siguientes, $row["materia"]);
261     }
262 }
263
264 // Iterar sobre materias del semestre
265 $materiasBySemestre = array(); // Inicializamos el arreglo para organizar las materias por semestre
266
267 foreach ($materias as $materia) {
268     // Agrupamos las claves de materias por semestre
269     $materiasBySemestre[$materia["semestre_materia"]][] = $materia["nombre_materia"];
270 }
271
272

```

- Consulta para obtener materias recomendadas para el siguiente semestre.
 - Join entre las tablas kardex y \$seriaciones: Se están uniendo las tablas kardex (donde están los registros de las materias) y \$seriaciones (que contiene información de las materias que tienen prerequisites) usando el campo materia en kardex y nombreMateria en \$seriaciones.
 - Filtrado por matrícula: Solo se obtienen los registros del alumno cuya matrícula coincide con la proporcionada.
 - Exclusión de materias cursadas: Se usa la condición NOT IN ('\$cursadas_str') para excluir las materias que ya están en la lista de materias cursadas.
 - Condiciones sobre los prerequisites: Se asegura de que el prerequisite de la materia esté entre las materias cursadas o que no haya prerequisite (nombrePre = '-').
 - Condiciones sobre el periodo y el estado de la materia: La consulta también incluye algunas condiciones para filtrar las materias dependiendo de si están registradas en el periodo o si el estado es "-".
 - Limitación del número de resultados: Se limita a un máximo de 7 materias.

Guardar materias siguientes en el arreglo \$siguientes

Después de ejecutar la consulta, se recorren los resultados y se guardan las materias siguientes en el arreglo \$siguientes. Estas son materias que aún no ha cursado el alumno, pero que son necesarias para su plan de estudios.

- Agrupar las materias por semestre ○ Este bloque agrupa las materias del alumno por semestre.

El arreglo

\$materiasBySemestre tendrá las claves como el número de semestre y los valores como los nombres de las materias de ese semestre.

- Esto se realiza iterando sobre las materias y organizándolas en un arreglo multidimensional.

```

274 // Inicializar arreglos para colores, calificaciones y créditos
275 $color = array();
276 $calificaciones = array();
277 $creditos = array();
278
279 $sql = "SELECT REPLACE(materia, ':', '') AS materia_limpia, cf, creditos
280 FROM kardex
281 WHERE status='Aprobado' AND matricula='".$matricula_alumno."'";
282 $result = $conn->query($sql);
283 if ($result->num_rows > 0) {
284     // Recorre cada fila de resultados
285     while ($row = $result->fetch_assoc()) {
286         $calificaciones[$row["materia_limpia"]] = $row["cf"]; // Asociamos el nombre de la materia con la calificación
287         $creditos[$row["materia_limpia"]] = $row["creditos"];
288     }
289 }
290
291 foreach ($materiasBySemestre as $semestre => $materiasDelSemestre) {
292     foreach ($materiasDelSemestre as $nombreMateria) {
293         // Normalizar el nombre de la materia
294         $nombreMateriaNormalizado = normalizarTexto($nombreMateria);
295
296         if (in_array($nombreMateriaNormalizado, array_map('normalizarTexto', $aprobadas))) {
297             $color[$semestre][$nombreMateriaNormalizado] = 'white'; // Materias aprobadas
298         } elseif (in_array($nombreMateriaNormalizado, array_map('normalizarTexto', $actuales))) {
299             $color[$semestre][$nombreMateriaNormalizado] = 'orange'; // Materias actuales
300         } elseif (in_array($nombreMateriaNormalizado, array_map('normalizarTexto', $siguientes))) {
301             $color[$semestre][$nombreMateriaNormalizado] = 'blue'; // Materias disponibles
302         } else {
303             $color[$semestre][$nombreMateriaNormalizado] = 'yellow'; // Materias no aprobadas
304         }
305     }
306 }

```

- Consulta SQL para obtener las calificaciones y créditos de materias aprobadas.
 - La consulta selecciona las materias aprobadas (status='Aprobado') para la matrícula especificada.
 - Usa REPLACE(materia, ':', '') para eliminar los dos puntos de los nombres de las materias. ○ Devuelve las calificaciones (cf) y créditos de esas materias.
 - Se recorren los resultados de la consulta y se llenan los arreglos \$calificaciones y \$creditos con los valores correspondientes a las materias.
- Asignación de colores a las materias por semestre.
 - Iteración por semestres: Se recorre el arreglo \$materiasBySemestre, que contiene las materias agrupadas por semestre.
 - Normalización de los nombres de las materias: Se normaliza el nombre de cada materia usando la función normalizarTexto() para asegurar que la comparación sea consistente (eliminar tildes, convertir a mayúsculas, etc.).
 - Condicionales para asignar colores:

Programación web II, Manual técnico 05/12/2024 ◦

- Si la materia está en el arreglo \$aprobadas, se asigna el color 'white' (blanco).
- Si la materia está en el arreglo \$actuales, se asigna el color 'orange' (naranja). □
Si la materia está en el arreglo \$siguientes, se asigna el color 'blue' (azul).
- Si la materia no está en ninguno de los tres arreglos anteriores, se asigna el color 'yellow' (amarillo).

```
308 //normalizar los arreglos
309 foreach ($materiasBySemestre as $semestre => $materiasDelSemestre) {
310     // Normalizamos el nombre de cada materia del semestre
311     $materiasBySemestre[$semestre] = array_map('normalizarTexto', $materiasDelSemestre);
312 }
313
314 foreach ($materiasVeces as $nombreMateria => $vecesCursada) {
315     // Normalizamos el nombre de la materia (clave)
316     $nombreMateriaNormalizado = normalizarTexto($nombreMateria);
317
318     // Asignamos el valor (veces cursada) a la clave normalizada
319     $materiasVeces[$nombreMateriaNormalizado] = $vecesCursada;
320 }
321
322 // Recorremos los arreglos y normalizamos las claves
323 foreach ($calificaciones as $nombreMateria => $calificacion) {
324     // Normalizamos el nombre de la materia (clave)
325     $nombreMateriaNormalizado = normalizarTexto($nombreMateria);
326
327     // Asignamos el valor (calificación) a la clave normalizada
328     $calificaciones[$nombreMateriaNormalizado] = $calificacion;
329 }
330
331 foreach ($creditos as $nombreMateria => $credito) {
332     // Normalizamos el nombre de la materia (clave)
333     $nombreMateriaNormalizado = normalizarTexto($nombreMateria);
334
335     // Asignamos el valor (créditos) a la clave normalizada
336     $creditos[$nombreMateriaNormalizado] = $credito;
337 }
338 }
```

- Normalización de los arreglos de materias por semestre, las veces que se han cursado, calificaciones y créditos para poder construir las tarjetas comparando la información con los arreglos de las materias cursadas, siguientes y actuales.

```
340 echo "<div class='mapa-curricular'>";
341 foreach ($materiasBySemestre as $semestre => $materiasDelSemestre) {
342     echo "<div><h3 style='text-align:center'> SEMESTRE " . $semestre . "</h3></div>";
343     foreach ($materiasDelSemestre as $nombreMateria) {
344
345         $colorClase = $color[$semestre][$nombreMateria];
346
347         echo "<div class='card $colorClase'>";
348
349         echo "<div class='face front'>";
350
351         // Mostrar el nombre de la materia original (sin normalizar)
352         if (isset($materiasVeces[$nombreMateria])) {
353             $vecesCursada = $materiasVeces[$nombreMateria];
354             echo "<p style='font-weight: bold; padding: 10px; background-color: #f0f8ff; border-radius: 5px; color: #333; text-align:center'>";
355             Veces que se cursó: {$vecesCursada}
356             </p>";
357         } else {
358             echo "<p style='font-weight: bold; padding: 10px; background-color: #ffe6e6; border-radius: 5px; color: #333;'>";
359             Veces que se cursó: 0
360             </p>"; // Por si no está en el arreglo
361         }
362
363         // Imprimir el nombre de la materia original
364         echo "<h3 style='font-size: 15px'> " . strtoupper($nombreMateria) . "</h3>";
365         echo "</div>";
366
367         echo "<div class='face back'>";
368         $calificacion = $calificaciones[$nombreMateria];
369         $creditosObt = $creditos[$nombreMateria];
370
371         if ($calificacion == 0) $calificacion = "NA";
372         if ($creditosObt == 0) $creditosObt = "NA";
373
374         echo "<p style=' font-weight: bold;'>Calificación Obtenida: {$calificacion}</p><br>";
375         // Créditos (Si se tienen, puedes incluir esta lógica)
376         echo "<p style=' font-weight: bold;'>Créditos Obtenidos: {$creditosObt} </p>"; // Aquí podrías añadir lógica para mostrar créditos si están disponibles
377         echo "</div>";
378
379         echo "</div>";
380     }
381 }
```

- echo "<div class='mapa-curricular'>"; Inicia la contenedora principal del mapa curricular, que se llenará con la información de las materias organizadas por semestre.
- Iteración sobre semestres.

- Se recorre el arreglo \$materiasBySemestre, que tiene las materias agrupadas por semestre.

Por cada semestre, se muestra el encabezado correspondiente (SEMESTRE X).

- Tarjetas de materia.
 - Asocia un color a cada materia según su estado (aprobada, cursada, disponible, etc.) utilizando el arreglo \$color.
 - Se crea una tarjeta (card) con el color determinado por el estado de la materia.
- Cara frontal de la tarjeta (Información básica).
 - La cantidad de veces que la materia ha sido cursada.
 - El nombre de la materia en mayúsculas.
- Cara trasera de la tarjeta (Calificación y Créditos)
 - Calificación obtenida: Se obtiene de \$calificaciones.
 - Créditos obtenidos: Se obtiene de \$creditos
 - Si no hay calificación o créditos disponibles (0), se muestra "NA" (No aplicable).

10.2 CSS

```

1
2 @import url('https://fonts.googleapis.com/css2?family=Bebas+Neue&family=Fira+Sans:ital,wght@0,100;0,200;0,300;0,400;0,500;0,600;0,700;0,800;0,900;1,
3
4 * {
5   margin: 0;
6   padding: 0;
7   box-sizing: border-box;
8 }
9
10 body {
11   width: 100%;
12   height: 100vh;
13   display: flex;
14   flex-direction: column;
15   align-items: center; /* Alinea verticalmente */
16   background: linear-gradient(90deg, #4b6cb7 0%, #182848 100%);
17 }
18
19 .mapa-curricular {
20   background-color: #f3f3f3;
21   width: 90%;
22   height: auto;
23   display: grid;
24   grid-template-columns: repeat(7, 1fr); /* 7 columnas para las filas regulares */
25   border: 5px solid black;
26   background: rgb(238,174,202);
27   background: radial-gradient(circle, rgba(238,174,202,1) 0%, rgba(148,187,233,1) 100%);
28   align-items: center;
29   gap: 20px; /* Espacio entre las tarjetas */
30   padding: 20px; /* Añadir un poco de relleno interno */
31 }

```

- Estilo general del cuerpo. El cuerpo se ajusta para ocupar el 100% de la altura de la ventana (100vh). Usa un gradiente de color como fondo, pasando de un azul claro a uno más oscuro.
- Mapa curricular. Usa un diseño de rejilla (grid) para organizar las materias en 7 columnas. El fondo tiene un gradiente radial de tonos pastel. El contenedor también tiene un borde negro grueso que resalta las tarjetas dentro del mapa.

```

34 .white{
35   position: relative;
36   height: 200px;
37   border: 4px solid black;
38   background-color: #77dd77;
39   border-radius: 15px;
40 }
41
42 .orange{
43   position: relative;
44   height: 200px;
45   border: 4px solid black;
46   background-color: #fcb7af;
47   border-radius: 15px;
48 }
49
50 .blue{
51   position: relative;
52   height: 200px;
53   border: 4px solid black;
54   background-color: #84b6f4;
55   border-radius: 15px;
56 }
57
58
59 .yellow{
60   position: relative;
61   height: 200px;
62   border: 4px solid black;
63   background-color: #fdd996;
64   border-radius: 15px;
65 }
66
67

```

- Estilos para las tarjetas de las materias (colores según el estado).
 - White: Para materias aprobadas. ○
 - Orange: Para materias actuales. ○
 - Blue: Para materias sugeridas. ○
 - Yellow: Para materias que aún no han sido cursadas. ○
- Radio de bordes: Todas las tarjetas tienen bordes redondeados para un mejor diseño.

```

68 .card .face{
69   position: absolute;
70   width: 100%;
71   height: 100%;
72   backface-visibility: hidden;
73   border-radius: 10px;
74   overflow: hidden;
75   transition: .5s;
76 }
77
78 .card .front{
79   transform: perspective(600px) rotateY(0deg);
80   box-shadow: 0 5px 10px #000 ;
81 }
82
83 .card .front h3{
84   font-family: "Roboto", serif;
85   position: absolute;
86   text-align: center;
87   bottom: 0;
88   width: 100%;
89   height: 50px;
90   line-height: 25px;
91   color: #000000;
92   background: #97989a;
93 }
94
95 .card .back{
96   transform: perspective(600px) rotateY(180deg);
97   background: rgb(3,35,54);
98   padding: 40px;
99   color: #f3f3f3;
100  display: flex;
101  flex-direction: column;
102  justify-content: center;
103  text-align: center;

```

- Estilos para la animación de las tarjetas (flip)
 - Se utiliza un efecto de voltear (rotateY) para las tarjetas. Cuando el usuario pasa el mouse por encima (:hover), la tarjeta gira y revela la parte posterior.

- Parte delantera (front): Muestra información básica, como el nombre de la materia.
- Parte trasera (back): Muestra información adicional como la calificación y los créditos.

```

106 .card:hover .front{
107   transform: perspective(600px) rotateY(180deg);
108 }
109
110 .card:hover .back{
111   transform: perspective(600px) rotateY(360deg);
112 }
113
114 .encabezado{
115   margin: 50px;
116   width: 90%;
117   display: flex;
118   justify-content: space-between;
119 }
120
121 .logo{
122   background-color: #f3f3f3;
123   border-radius: 10px;
124   padding: 10px;
125   height: 100px;
126   width: 200px;
127 }
128
129 .titulo{
130   display: flex;
131   flex-direction: column;
132   align-items: center;
133   justify-content: center;
134 }

```

- Estilo para el encabezado. Tiene un margen amplio y usa un diseño de flexbox para organizar los elementos (como el logo y el título) de manera eficiente.
- Estilo del logo y título.
 - Logo: Un estilo simple pero efectivo para el logo, con bordes redondeados y un fondo claro.
 - Título: El nombre de la universidad tiene un estilo destacado con un efecto de sombra para hacerlo más llamativo.

```

136 .uni{
137   font-family: "Oswald", sans-serif;
138   font-size: 1.5em;
139   font-weight: bold;
140   text-align: center; /* Centrado */
141   color: #fff; /* Color blanco para el texto */
142   text-transform: uppercase; /* Convierte a mayúsculas */
143   letter-spacing: 2px; /* Espaciado entre letras */
144   border-radius: 10px; /* Bordes redondeados */
145   text-shadow: 3px 3px 5px rgba(0, 0, 0, 0.3), 0 0 25px rgba(0, 0, 255, 0.5), 0 0 5px rgba(255, 255, 255, 0.5); /* Sombra de texto para dar profundidad */
146   transition: transform 0.3s ease-in-out, color 0.3s ease;
147 }
148
149 .datos-alumno{
150   font-family: "Arial", sans-serif; /* Fuente clara y legible */
151   font-size: 2.5em; /* Tamaño de fuente adecuado */
152   font-weight: 400; /* Peso normal */
153   color: #333; /* Color de texto oscuro para una buena legibilidad */
154   text-align: center; /* Centrado */
155   padding: 10px 20px; /* Espaciado interno */
156   background-color: #cccccc; /* Fondo claro para contraste */
157   border-radius: 5px; /* Bordes redondeados */
158   margin: 20px auto; /* Márgenes alrededor para separar el subtítulo */
159   letter-spacing: 0.5px; /* Ligero espaciado entre letras */
160   text-transform: capitalize; /* Primera letra en mayúscula */
161 }
162
163 .leyendascont {
164   color: white;
165   display: flex;
166   justify-content: space-around;
167   padding: 20px;
168   margin: 50px;
169   background-color: #f8f8f8;
170 }
171

```

- Estilo de la sección de datos del alumno. El estilo es claro y legible, con un fondo claro y bordes redondeados. El texto se centra para facilitar la lectura.
- Leyendas de colores. Se utiliza un diseño flexible para mostrar leyendas de color que explican el significado de cada color en el mapa curricular (aprobadas, no cursadas, actuales, sugeridas).

```
172 .leyendas {
173     display: flex;
174     align-items: center;
175 }
176
177 .cl {
178     width: 20px;
179     height: 20px;
180     margin: 10px;
181     border-radius: 50%;
182 }
183
184 #aprobadas{
185     background-color: #77dd77;
186 }
187
188 #sin-cursar {
189     background-color: #fdfd96;
190 }
191
192 #actualmente {
193     background-color: #fcb7af;
194 }
195
196 #sugeridas {
197     background-color: #84b6f4;
198 }
199
200 .leyenda {
201     font-size: 16px;
202     font-weight: bold;
203 }
```

11. PAGINA ALUMNO.PHP

Propósito: Habilita el uso de sesiones para identificar usuarios y almacenar datos persistentes durante la navegación. Uso sugerido:

- Verificar si existe una sesión activa al inicio de la página para limitar el acceso a usuarios no autenticados:

```
<?php
    session_start();
?>
```

- Cabecera (<head>):
- Define el idioma (lang="es") para mejorar la accesibilidad y SEO.
- Incluye los archivos de estilo CSS para personalizar el diseño:
 - alumnos.css: Estilo general de la página.
 - navM.css: Estilo específico para la barra de navegación.

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Alumnos</title>
  <link rel="stylesheet" href="./styles/alumnos.css">
  <link rel="stylesheet" href="./styles/navM.css">
</head>
```

Una barra de navegación con una clase barra para personalización.
Tres opciones:

- Inicio: Enlace a la página principal paginaAl.php.
- Logout: Redirige al script logout.php para cerrar la sesión.
- Help: Enlace a la página de ayuda ayuda.php.

El atributo class="active" indica el enlace actualmente seleccionado.

```
<body>
  <!-- Barra de navegación -->
  <nav class="barra">
    <ul class="topnav">
      <li><a class="active" href="./paginaAl.php">Inicio</a></li>
      <li><a href="./archivosPHP/logout.php">Logout</a></li>
      <li class="right"><a href="./archivosPHP/ayuda.php">Help</a></li>
    </ul>
  </nav>
```

El contenedor principal usa la clase card-container para organizar las opciones en forma de tarjetas.

Cada tarjeta tiene:

- Un título (<h2>) que describe la funcionalidad.
- Un enlace (<a>):
 - Ver Kardex: Carga la página verKardexAlumno.php para mostrar información en línea.

```
<div class="card-container">
  <div class="card">
    <h2>Ver Kardex</h2>
    <a href="./archivosPHP/verKardexAlumno.php">Ver KARDEX</a>
  </div>

  <div class="card">
    <h2>Imprimir Kardex</h2>
    <a href="#" onclick="window.open('./archivosPHP/kardexPDF.php', '_blank');">Imprimir KARDEX</a>
  </div>
</div>
</body>
</html>
```

- Imprimir Kardex: Abre kardexPDF.php en una nueva pestaña utilizando window.open().

12. PÁGINA DE MAESTROS

12.1. HTML

```

<?php
<doctype html>
<html lang="es">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Profesores</title>
<link rel="stylesheet" href="./styles/paginaMaestros.css">
<link rel="stylesheet" href="./styles/navM.css">
</head>
<body>
<!-- Barra de navegación -->
<nav class="barra">
<ul class="topnav">
<li><a class="active" href="./paginaM.html">Inicio</a></li>
<li><a href="./archivosPHP/logout.php">Logout</a></li>
<li class="right"><a href="./archivosPHP/ayuda.php">Help</a></li>
</ul>
</nav>

<header>¡Bienvenido a la Página de Profesores!</header>

<!-- Contenedor de tarjetas -->
<div class="card-container">
<!-- Tarjeta 1 -->
<div class="card">
<h2>Kardex de alumno</h2>
<p>Accede al kardex de los alumnos.</p>
<a href="./archivosPHP/verKardex.php">Ir a Kardex</a>
</div>

<!-- Tarjeta 2 -->
<div class="card">
<h2>Reporte del alumno</h2>
<p>PDF para la descarga del mapa consultado.</p>
<a href="./archivosPHP/reporte.php">Ir a Reporte</a>
</div>

<!-- Tarjeta 3 -->
<div class="card">
<h2>Estadísticas de alumno</h2>
<p>Imprime un reporte con el % de avance de cada estudiante.</p>
<a href="./archivosPHP/estadistica_control.php">Ir a Estadísticas</a>
</div>
</div>
</body>

```

- `<nav class="barra">`
Este elemento define una barra de navegación que permite a los usuarios desplazarse entre las principales funcionalidades de la página. Contiene una lista `<ul>` con enlaces a las páginas: "Inicio", "Logout" y "Help". La clase barra se utiliza para estilizar el componente.
- `<li><a class="active" href="./paginaM.html">Inicio</a></li>`
Este elemento de la lista representa el enlace a la página de inicio. La clase active indica que es la página actual, generalmente resaltada visualmente.
- `<header>`

Este elemento contiene un mensaje de bienvenida: "¡Bienvenido a la Página de Profesores!", que sirve como introducción al contenido de la página.

- `<div class="card-container">`
Este contenedor agrupa todas las tarjetas que representan las distintas opciones funcionales disponibles para los profesores.
- `<div class="card">`
Este elemento define una tarjeta individual. Cada tarjeta incluye un título, una breve descripción y un enlace.
- `<h2>Kardex de alumno</h2>`
Este encabezado dentro de la tarjeta proporciona el título: "Kardex de alumno", que describe el propósito de la tarjeta.
- `<p>Accede al kardex de los alumnos. </p>`
Este párrafo explica brevemente la funcionalidad asociada a esta tarjeta.
- `<a href="/archivosPHP/verKardex.php">Ir a Kardex</a>`
Este enlace permite a los usuarios acceder al módulo de consulta del kardex de los alumnos.
- `<div class="card">` (segunda y tercera instancia)
Estas tarjetas siguen el mismo formato que la primera, pero están adaptadas para diferentes funcionalidades:

Reporte del alumno: Generar un PDF con información académica.

Estadísticas de alumno: Obtener un reporte del avance académico en porcentaje.

12.2 CSS

```
# paginaMaestros.css X
styles > # paginaMaestros.css > body
1  /* General styles */
2  * {
3      margin: 0;
4      padding: 0;
5      box-sizing: border-box;
6  }
7
8  body {
9      font-family: 'Arial', sans-serif;
10     background-color: #f7f7f7;
11     color: #333;
12     line-height: 1.6;
13     margin: 0;
14     background: linear-gradient(90deg, #4b6cb7 0%, #182848 100%);
15 }
16
17 /* Header */
18 header {
19     text-align: center;
20     font-size: 32px;
21     font-weight: bold;
22     margin-top: 30px;
23     color: #ffffff;
24     text-transform: uppercase;
25     letter-spacing: 2px;
26 }
27
28 /* Card container */
29 .card-container {
30     display: flex;
```

- *

Se aplica a todos los elementos para eliminar márgenes y padding por defecto y asegurar un diseño uniforme.

- box-sizing: border-box: Asegura que el ancho/alto de los elementos incluya padding y bordes.

- body

Establece los estilos globales de la página:

- Fuente: Arial, simple y legible.
- Fondo: Gradiente lineal con transición entre dos tonos (#4b6cb7 a #182848).
- Color de texto: Gris oscuro (#333), para buen contraste y legibilidad.

Estilo del Encabezado (header)

- Centra el texto del encabezado principal con text-align: center.
- Aplica estilos visuales como:
 - Mayúsculas automáticas (text-transform: uppercase).
 - Espaciado entre letras (letter-spacing: 2px).
 - Color blanco (#ffffff) para destacar sobre el fondo oscuro.

```
29 .card-container {
30
31     flex-wrap: wrap;
32     justify-content: center;
33     margin: 50px auto;
34     gap: 20px;
35     padding: 0 20px;
36 }
37
38 /* Cards */
39 .card {
40     flex-direction: column;
41     background-color: rgba(255, 250, 250, 0.785);
42     width: 300px;
43     padding: 20px;
44     border-radius: 10px;
45     box-shadow: 0 8px 15px rgba(0, 0, 0, 0.1);
46     text-align: center;
47     transition: transform 0.3s ease, box-shadow 0.3s ease;
48 }
49
50 .card:hover {
51     transform: translateY(-10px);
52     box-shadow: 0 12px 20px rgba(0, 0, 0, 0.2);
53 }
54
55 .card h2 {
56     font-size: 24px;
57     color: #333;
58     margin-bottom: 15px;
59 }
```

Estilos Generales

Estilos de Contenedor de Tarjetas (.card-container)

- Uso de Flexbox:
 - display: flex; flex-wrap: wrap; justify-content: center: Dispone las tarjetas en filas centradas y permite que pasen a la siguiente línea si no caben.
- Espaciado:
 - gap: 20px: Espacio uniforme entre las tarjetas.
 - padding: 0 20px: Margen interno para evitar que el contenido esté pegado a los bordes.

Estilos de Tarjetas (.card)

- Apariencia:
 - Fondo semitransparente con color claro.
 - Bordes redondeados (border-radius: 10px) y sombra (box-shadow) para un efecto de profundidad.
 - Textos centrados con text-align: center.
- Transiciones:

- Animaciones suaves al pasar el cursor:
 - Hover (:hover): Eleva visualmente la tarjeta y aumenta la sombra, simulando un efecto flotante (transform: translateY(-10px)).

Textos dentro de Tarjetas (.card h2 y .card p)

- h2:
 - Tamaño: 24px. ○ Color: Gris oscuro (#333).
 - Espaciado inferior para separación del contenido.
- p: ○ Tamaño más pequeño (16px) y color gris claro (#555).

Contenedor de Botones (.card-buttons)

- Organización con Flexbox:
 - Los botones están centrados horizontalmente y tienen un espacio uniforme entre ellos (gap: 10px). ○ Se asegura una separación adecuada del contenido superior mediante margin-top: 20px.

```
styles > # paginaMaestros.css > body
55 .card h2 {
57   color: #333;
58   margin-bottom: 15px;
59 }
60
61 .card p {
62   font-size: 16px;
63   color: #555;
64   margin-bottom: 20px;
65 }
66
67 /* Card Buttons Container */
68 .card-buttons {
69   display: flex; /* Configura flexbox */
70   justify-content: center; /* Centra los botones horizontalmente */
71   gap: 10px; /* Espacio entre botones */
72   margin-top: 20px; /* Separación del contenido superior */
73 }
74
75 /* Botones dentro de las tarjetas */
76 .card a {
77   display: inline-block;
78   text-decoration: none;
79   color: white;
80   background-color: #e77817;
81   padding: 10px 20px;
82   border-radius: 5px;
83   font-size: 18px;
84   transition: background-color 0.3s ease;
85   flex: 1; /* Los hace del mismo ancho dentro de su fila */
```

Botones de las Tarjetas (.card a)

- Apariencia:
 - Fondo naranja (#e77817), texto blanco, y bordes redondeados. ◦ flex: 1:
 - Los botones tienen el mismo ancho dentro de su fila.
 - min-width: 100px; max-width: 200px: Asegura tamaños consistentes y evita deformaciones.
- Transiciones:
 - Efecto al pasar el cursor: Cambio de color del fondo a un tono más oscuro (#cf6407), lo que brinda retroalimentación visual al usuario.

```

88 |   max-width: 200px; /* Opcional: Define un ancho máximo */
89 | }
90 |
91 | /* Hover para los botones */
92 | .card a:hover {
93 |   background-color: #cf6407;
94 | }
95 |
96 |

```

13. STYLES

1. Admin.css

1. Importación de Fuentes

```
@import url('https://fonts.googleapis.com/css2?family=Montserrat:wght@300;400;600&display=swap');
```

Se utiliza la fuente *Montserrat*, importada desde Google Fonts. Se aplican diferentes pesos para estilizar los textos.

2. Reseteo y Configuración Global

Se define un reseteo básico para todos los elementos:

```

* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
  font-family: "Montserrat", sans-serif;
}

```

- Eliminación de márgenes y rellenos.
- Configuración del modelo de caja con boxsizing: border-box.

- Aplicación de la fuente global "Montserrat", sans-serif.

3. Estilo del Cuerpo (body)

Define el fondo, alineación, y disposición principal:

```
body {
  background-color: #f7f7f7;
  background-size: cover;
  background-repeat: no-repeat;
  color: #f38f02af;
  display: flex;
  flex-direction: column;
  align-items: center;
  min-height: 100vh;
}
```

- Fondo de color claro con tamaño adaptable.
- Texto de color semitransparente.
- Uso de flexbox para centrar y alinear elementos.
- Altura mínima de la pantalla.

4. Contenedor Principal (.contenido)

Un contenedor flexible y responsivo que aloja las cajas :

```
23 .contenido {
24   background: rgba(0, 89, 255, 0.336); /* Fondo transparente */
25   backdrop-filter: blur(5px);
26   border-radius: 15px;
27   padding: 40px;
28   width: 90%;
29   max-width: 1200px;
30   margin: 40px auto;
31   display: flex;
32   flex-wrap: wrap;
33   gap: 30px;
34   justify-content: space-around;
35   box-shadow: 0 6px 12px rgba(0, 0, 0, 0.1);
36 }
37
```

- Fondo translúcido con efecto de desenfoque.
- Bordes redondeados y sombras suaves.
- Disposición de los elementos en filas con flex-wrap para adaptarse al espacio disponible.

- Espaciado uniforme entre elementos.

5. Estilo de Cajas (.caja)

Configura un contenedor de filas flexibles para organizar elementos (.tipo):

```
/* Estilo de las cajas */
.caja {
  display: flex;
  flex-direction: row;
  gap: 30px;
  flex-wrap: wrap; /* Esto asegura que las cajas se ajusten en varias líneas si no caben */
}
```

- Permite múltiples líneas con flex-wrap.
- Espaciado uniforme entre los elementos secundarios.

6. Estilo de Elementos Secundarios (.tipo)

Define cajas individuales con diseño limpio y organizado:

```
.tipo {
  background-color: #ffffff;
  border: 2px solid #e77817;
  border-radius: 10px;
  padding: 20px;
  width: 300px;
  min-height: 200px;
  box-shadow: 0 6px 12px rgba(0, 0, 0, 0.1);
  display: flex;
  flex-direction: column;
  justify-content: space-between; /* Espacio entre el h5 y el enlace */
}
```

- Fondo blanco, bordes redondeados y bordes visibles.
- Sombra suave para profundidad.
- Disposición vertical con flexbox, asegurando espacio uniforme entre contenido

7. Botones de Acción (.accion)

Botones diseñados para interactividad y visibilidad:

```
.accion {  
  background-color: #152874;  
  color: #ffffff;  
  text-decoration: none;  
  padding: 10px 15px;  
  text-align: center;  
  border-radius: 5px;  
  transition: background-color 0.3s ease;  
}  
  
.accion:hover {  
  background-color: #e77817;  
}
```

- Fondo azul oscuro con texto blanco y bordes redondeados.
- Efecto hover que cambia el color de fondo a naranja.

8. Responsividad (@media):

Diseño adaptativo para pantallas pequeñas:

```
/* Responsivo */  
@media (max-width: 768px) {  
  .fotos {  
    flex-direction: column;  
    align-items: center;  
    gap: 20px;  
  }  
  
  .contenido {  
    flex-direction: column;  
    align-items: center;  
  }  
  
  .tipo {  
    width: 100%;  
    max-width: 400px;  
  }  
}
```

- Las cajas y el contenedor principal se reorganizan en una columna.
- Se ajusta el ancho de las cajas para maximizar el uso del espacio disponible.

14. CONCLUSIONES

El sistema desarrollado proporciona un ejemplo funcional de cómo combinar tecnologías del lado del servidor y del cliente para construir una aplicación web robusta y segura. La estructura modular del código permite una fácil extensión y mantenimiento, mientras que el diseño responsivo asegura una experiencia de usuario óptima en diversos dispositivos.

A través de este manual técnico, se ofrecen guías claras y prácticas para comprender la lógica detrás del sistema, desde la gestión de sesiones y roles de usuario hasta la presentación visual de la interfaz. Esto no solo facilita el uso del sistema actual, sino que también sienta las bases para futuros desarrollos y mejoras. Con el enfoque en la claridad y la escalabilidad, este proyecto puede ser un recurso valioso tanto para principiantes como para desarrolladores experimentados.